Motor antifraude de Pix: relatório técnico

Hackathon QI Tech 2026 · Risk Solutions

Pedro H. S. Prestes Laura Pazini Medeiros

18 de outubro de 2026

Sumário

Resumo executivo **3**

1 O problema **4**
1.1 O Pix e a decisão em tempo real . . . 4
1.2 As três decisões . . . 4
1.3 Fricção × fraude . . . 4
1.4 O MED e o atraso do rótulo . . . 4
1.5 O enunciado . . . 5

2 Fundamentos teóricos **6**
2.1 Detecção de fraude: regras × modelos e os níveis de Bolton & Hand . . . 6
2.2 Estatística robusta: mediana, MAD e z robusto sobre $\log(\text{valor})$ . . . 6
2.3 Contagens: Poisson, conjugada Gamma e a preditiva binomial negativa . . . 7
2.4 Distância de haversine e viagem impossível . . . 7
2.5 Tipologias de golpe e regras compostas . . . 8
2.6 Grafos: funil, dispersão, vizinhança de dois saltos e GraphSAGE . . . 8
2.7 Sequências: HMM discreto, Baum–Welch e verossimilhança como anomalia . . . 9
2.8 Regressão logística, calibração de Platt, *gradient boosting* e Isolation Forest . . . 9
2.9 Desbalanceamento, custo assimétrico e P@k . . . 9
2.10 PSI e deriva; rótulos atrasados; viés de seleção; *champion/challenger* e *shadow mode* . . . 10

3 Arquitetura **11**
3.1 Camadas do Base Service . . . 11
3.2 *Hot path* × *cold path* . . . 11
3.3 Diagramas C4 . . . 11
3.4 Contrato da API . . . 16
3.5 Modelo de dados . . . 16

4 Implementação **17**
4.1 O motor: *snapshot*, contexto, `HistoryPort` e decisão . . . 17
4.2 Catálogo de regras, parâmetros e justificativas . . . 17
4.2.1 Limite: “passou do teto?” . . . 17
4.2.2 Velocity: “rápido demais?” . . . 18
4.2.3 Montante: “somou demais?” . . . 18
4.2.4 Perfil: “está no padrão do cliente?” . . . 18
4.2.5 Contraparte: “quem recebe é confiável?” . . . 19
4.2.6 Tipologia: combinações que descrevem um golpe . . . 19
4.2.7 Regras descartadas . . . 20

4.3 Modelos servidos em Python puro . . . 20
4.4 O worker (*cold path*) . . . 21
4.5 Backtest em lote e paridade . . . 21
4.6 Mapa do código . . . 21

5 Metodologia de validação **23**
5.1 Por que o nosso simulador é só *fixture* de CI . . . 23
5.2 O PixSim . . . 23
5.3 Escala reduzida . . . 23
5.4 Adaptador: o que foi mapeado, inventado e não existe . . . 24
5.5 Mundos A e B; perspectiva do banco pagador . . . 24
5.6 Métricas . . . 24
5.7 Avaliação nos dois sentidos . . . 25
5.8 Calibração por custo (mundo A escolhe, mundo B confirma) . . . 25
5.9 Alinhamento com a metodologia do PixSim e comparação com o artigo . . . 25
5.10 Vazamento de dados e vieses do gerador . . . 26
5.11 O que o PixSim não gera . . . 26

6 Resultados no PixSim **28**
6.1 Fricção em legítimos . . . 28
6.2 Captura por tipologia . . . 28
6.3 Custo esperado . . . 28
6.4 P@k diário . . . 29
6.5 Calibração: efeito de cada parâmetro e configuração final . . . 29
6.6 Modelos em *shadow* . . . 30
6.7 Rótulos atrasados . . . 31
6.8 Retroalimentação pelo MED . . . 32
6.9 Peer group: medido e descartado . . . 32
6.10 Latência . . . 32
6.11 Deriva das características . . . 34
6.12 Contribuição de cada regra (ablação) . . . 34
6.13 Cascata de valor . . . 35
6.14 Sensibilidade à qualidade do escorer . . . 36

7 Limitações e trabalhos futuros **37**

8 Como reproduzir **38**

9 Perguntas prováveis da banca **39**
9.1 O problema e a decisão . . . 39
9.2 Regras e calibração . . . 39
9.3 Validação . . . 40
9.4 Modelos e retroalimentação . . . 41
9.5 Engenharia . . . 41
9.6 Próximos passos e processo . . . 41

Referências **42**

Resumo executivo

Problema. O desafio do Hackathon QI Tech 2026 pede uma API que receba um Pix antes de ele sair e responda, com motivo, uma de três decisões: `approved`, `two_factor_required` (2FA) ou `rejected`. Não há rótulo de fraude, nem analista, nem consulta externa no caminho da decisão. Bloquear cliente bom também é prejuízo; deixar passar fraude custa o valor do Pix.

Solução. Um motor de regras explícitas e explicáveis sobre o histórico que a própria API acumula (Python 3.11, FastAPI, PostgreSQL 16, Docker Compose). Cada Pix atravessa 15 regras (limite, velocidade, perfil, contraparte e tipologias de golpe), e a decisão final é a mais severa entre as que dispararam; todos os disparos vão para `metadata.reasons`, com os números usados em `metadata.signals`. Sinais estatísticos (z robusto de $\log(\text{valor})$, preditiva Poisson–Gamma, haversine) entram como regras comuns, em Python puro. Um segundo caminho, assíncrono (*cold path*), calcula a reputação de rede de cada conta de destino (funil, dispersão, vizinhança de dois saltos) e dois modelos aprendidos (GraphSAGE e HMM). Cinco modelos rodam em *shadow*: calculam, gravam e nunca mudam a decisão.

Como validamos. O nosso simulador gera as fraudes que nós mesmos desenhamos, e as regras procuram exatamente esses padrões; sobre ele a captura chegava a 98,5%, o que é circular. Por isso toda métrica deste relatório vem do PixSim [5], um simulador de terceiros, aberto e calibrado com dados abertos do Banco Central. O nosso simulador ficou como *fixture* de CI. Os parâmetros foram escolhidos em um mundo do PixSim (A) pelo custo esperado ponderado e confirmados em outro (B).

Números principais (PixSim, mundo A, 30 dias avaliados, 125.518 Pix legítimos, 986 eventos de fraude; *fonte:* `validacao-pixsim.md` *e* `calibracao.md`):

Métrica	Valor
Legítimos aprovados / 2FA / reprovados	97,03% / 2,97% / 0,00%
Pix de vítima barrados (2FA ou reprovação)	50,7%
Eventos de fraude com ao menos um Pix de vítima barrado	584 de 986 (conta tomada 98,6%; engenharia social 56,6%; QR adulterado 27,9%; Pix errado 9,9%)
Custo ponderado por 1.000 Pix (falso bloqueio a R$ 50 + 2%)	R$ 109,61 (mundo A); R$ 118,82 (mundo B), contra R$ 127,26 antes da calibração e R$ 153,51 com as 18 regras de antes do descarte (mundo B)
Latência do motor em memória (backtest)	P50 0,33 ms, P95 0,60 ms, P99 0,84 ms (2.710 Pix/s)
Modelos treinados no PixSim A, testados no B (AUC)	regressão logística 0,990; LightGBM 0,989; GraphSAGE de nó 0,92
Regras + LightGBM em *shadow* pedindo 2FA (mundo B)	80,9% dos Pix de vítima barrados; R$ 99,42 por 1.000 Pix

Limitações. Tudo é dado sintético: "métricas no PixSim" medem fricção e custo *relativo* entre configurações, não eficácia em produção. Os mundos A e B vêm do mesmo gerador (só muda a semente). O PixSim não tem geolocalização, então as regras geográficas ficam inertes na validação. A escala é reduzida (5.025 contas). Os rótulos de produção chegariam com atraso pelo MED, e a rota de desfechos não faz parte do contrato do enunciado. A seção 7 detalha o que falta.

1 O problema

1.1 O Pix e a decisão em tempo real

O Pix é o sistema de pagamentos instantâneos do Banco Central do Brasil (BCB). A liquidação é praticamente imediata e irreversível do ponto de vista do pagador, o que desloca a defesa para *antes* da saída do dinheiro: a instituição pagadora precisa decidir em segundos, com o cliente esperando, sem analista humano e sem consulta externa pesada [1]. Três tensões moldam o desenho:

Tensão	Consequência no desenho
Tempo. A decisão sai em poucos segundos.	Tudo o que a regra precisa está no payload ou no banco próprio. Nenhuma chamada HTTP no caminho da decisão (ADR 0002).
Histórico. Um Pix sozinho diz pouco; o sinal está na comparação com o passado do pagador, do destino e do aparelho.	Todo Pix recebido é gravado, inclusive os reprovados, com índices por janela de tempo (ADR 0009).
Fricção. Bloquear cliente bom é prejuízo.	Existe o meio-termo `two_factor_required`; cada parâmetro é calibrado e justificado.

1.2 As três decisões

Status	Significado	Quando
`approved`	O Pix segue sem fricção.	Nenhuma regra disparou.
`two_factor_required`	O cliente confirma que é ele (2FA).	Há dúvida: o sinal isolado ocorre também com cliente bom.
`rejected`	O Pix não sai.	O risco é claro: sinal extremo ou tipologia de golpe.

Um 2FA custa pouco ao cliente legítimo (um toque) e é eficaz contra quem não tem o segundo fator, mas é ineficaz quando a própria vítima, coagida ou enganada, confirma: é o caso da engenharia social, em que o dispositivo e as credenciais são legítimos. Por isso o desenho olha também a *conta de destino*, não apenas o pagador.

1.3 Fricção $\times$ fraude

O problema é de decisão sob custo assimétrico: uma fraude aprovada custa o valor do Pix; um cliente bom reprovado custa atendimento, abandono e *churn*; um 2FA a um cliente bom custa atrito pequeno. Acurácia é inútil quando mais de 99,9% das transações são legítimas (um classificador que responde "tudo legítimo" acerta 99,9%). Daí a escolha de métricas deste trabalho: custo esperado em reais e precisão nos k alertas mais suspeitos do dia (P@k), na seção 5.6.

1.4 O MED e o atraso do rótulo

O Mecanismo Especial de Devolução (MED) é o processo pelo qual a vítima contesta um Pix e o valor pode ser recuperado. Para a ciência de dados, o MED tem uma consequência: o *rótulo* de fraude só existe depois que a vítima reporta, dias depois do Pix. No PixSim, o atraso de reporte nas 1.380 fraudes de vítima do mundo A tem mediana de 2,1 dias e p90 de 17,5 dias, e 1,8% nunca são reportadas (*fonte:* `validacao-pixsim.md`*, "Rótulos atrasados"*). Modelos treinados em produção veem, no instante do treino, as fraudes mais recentes ainda rotuladas como legítimas; a seção 2.10 e a seção 6.7 tratam do efeito.

1.5 O enunciado

O desafio (transcrição fiel em `docs/01-contexto/enunciado.md`) pede: `POST /pix/transaction` recebe o Pix e devolve exatamente `{key, status, metadata}`, com `status` em `approved`, `rejected` ou `two_factor_required` e `metadata` livre; `GET /pix/transaction/{id}` devolve a análise gravada. Entram o banco próprio com o histórico de tudo o que chegou, as regras e os testes, o README e o ponto de partida do Base Service; ficam de fora onboarding, front-end e outras APIs. A política antifraude é livre, "desde que expliquem o porquê". O payload traz valor em centavos, data com fuso, conta de origem e destino (agência, conta, data de abertura, documento do titular, ISPB do destino), aparelho, canal, IP e geolocalização opcional. Não há rótulo de fraude.

Garantias que o time assumiu (`docs/01-contexto/problema.md`): todo Pix recebido fica gravado e consultável; o mesmo Pix enviado duas vezes não gera duas análises; a decisão é reprodutível (mesmo histórico e mesmo Pix, mesma decisão); dinheiro é inteiro em centavos; nenhuma decisão sai sem motivo. Ficam fora do escopo mover o dinheiro, o fluxo do 2FA, regras configuráveis sem código, modelo de ML decidindo (os modelos rodam só em *shadow*), consultas a bases externas e a retroalimentação na API: a API não recebe desfechos (MED, contestação, resultado do 2FA), porque o contrato do enunciado só tem `POST` e `GET`. A retroalimentação é simulada no backtest sobre o PixSim (seção 6.8), e a rota de desfechos fica como primeiro passo de produção (ADR 0032).

2 Fundamentos teóricos

Cada subseção responde a três perguntas: o que é o método, por que serve aqui e qual é a limitação. As referências numeradas estão na seção "Referências"; cada método cita a fonte primária ([3], [14] a [20]); a pesquisa interna do time [2] foi o ponto de partida, e as decisões e medições do projeto estão no próprio repositório [13].

2.1 Detecção de fraude: regras × modelos e os níveis de Bolton & Hand

O que é. Bolton e Hand [3] revisam a detecção estatística de fraude (métodos supervisionados e não supervisionados, *peer group analysis* e *break point analysis*); a pesquisa do time organiza essas ideias em quatro níveis de comparação:

1. *individual*: o comportamento histórico do próprio cliente (valores típicos, locais frequentes);
2. *grupo de pares* (*peer group analysis*): o cliente contra clientes semelhantes;
3. *ponto de ruptura* (*break point*): o instante em que o padrão de uma conta muda abruptamente;
4. *rótulos externos*: listas de entidades marcadas como suspeitas.

Por que serve aqui. Sem rótulos, a abordagem é não supervisionada ou de limiar sobre desvio do perfil. O nível 1 vira `PROFILE_AMOUNT_ROBUST_Z` e `VELOCITY_POISSON_GAMMA`. O nível 3 aparece só por aproximação: `VELOCITY_POISSON_GAMMA` compara a rajada recente com a taxa histórica do próprio pagador, `PROFILE_NEW_DEVICE` reage a uma mudança brusca de aparelho e o HMM de sequência (em *shadow*) mede a improbabilidade da sequência recente; uma análise de ponto de ruptura formal (duas janelas ou CUSUM) não foi implementada, e a conta tomada é uma combinação de sinais (ADR 0018), não uma detecção de ruptura. o nível 4, a reputação *derivada* do próprio histórico, sem lista externa, foi implementado como `COUNTERPARTY_FLAGGED` e descartado pela ablação (seção 4.2.7); a versão com desfecho confirmado pelo MED (`COUNTERPARTY_CONFIRMED_FRAUD`) existe só no backtest (seção 6.8). O nível 2 também foi implementado, medido e descartado (seção 6.9).

Regras × modelos. Regras são explícitas, auditáveis e dão um motivo por disparo, o que o enunciado exige; modelos de *gradient boosting* continuam o padrão da indústria em dados tabulares, mas precisam de rótulos. A pesquisa recomenda a pilha em camadas: regras e listas na primeira linha (0,5 a 2 ms), *ensembles* no caminho síncrono (2 a 8 ms), redes sequenciais e de grafos fora dele. Este projeto adota essa separação com uma diferença: as regras decidem e os modelos, inclusive os do caminho síncrono, ficam em sombra.

Limitação. Regras codificam hipóteses nossas; sem rótulo real não se mede sua taxa de erro em produção, só sobre simuladores.

2.2 Estatística robusta: mediana, MAD e z robusto sobre $\log(\text{valor})$

O que é. Para uma amostra $x_1, \ldots, x_n$, a mediana m e o desvio absoluto mediano $\text{MAD} = \text{mediana}_i\,|x_i - m|$ são estimadores de posição e escala com ponto de ruptura de 50%: metade das observações pode ser arbitrária sem derrubá-los. Multiplicado por $1{,}4826 = 1/\Phi^{-1}(3/4)$, o MAD é consistente com o desvio-padrão de uma normal. O z robusto de um valor a do pagador é

$$z = \frac{\ln a - m}{\max(1{,}4826 \cdot \text{MAD},\ s_{\min})}, \qquad m,\ \text{MAD calculados sobre } \{\ln a_i\} \text{ dos últimos 30 dias,}$$

com piso de escala $s_{\min} = 0{,}25$ (`RULE_PROFILE_MIN_SCALE_PERCENT = 25`) e histórico mínimo de 5 Pix (implementação em `robust_z` e `amount_z`).

Por que serve aqui. Valores de Pix são multiplicativos ("o dobro do usual" pesa igual em R$ 20 e em R$ 2.000), então o logaritmo simetriza. O histórico de cada pagador é pequeno e um Pix grande antigo não pode inflar a escala e esconder o próximo, o que acontece com média e desvio-padrão. O piso evita divisão por zero quando o histórico é idêntico (MAD = 0). A análise 1 de `analises.md` mediu, no mundo A do

PixSim, assimetria mediana de 0,03 de $\ln(\text{valor})$ por pagador, 85,4% dos 3.192 pagadores com |assimetria| $< 0{,}5$ e QQ a no máximo 0,01 da normal (em unidades de z), o que sustenta o uso do logaritmo.

Limitação. Pagador sem histórico (menos de 5 Pix em 30 dias) não tem z; ele fica coberto pelas regras de payload e de velocidade fixa. O grupo de pares que o cobriria foi descartado por não render captura no PixSim.

2.3 Contagens: Poisson, conjugada Gamma e a preditiva binomial negativa

Fonte primária: Gelman et al. [20] (conjugada Gamma–Poisson).

O que é. Se as chegadas de Pix de um pagador seguem um processo de Poisson de taxa λ (por hora), a contagem em uma janela de w horas é $N \mid \lambda \sim \text{Poisson}(\lambda w)$. A taxa é incerta; com prior conjugado $\lambda \sim \text{Gamma}(\alpha, \beta)$ (forma e taxa) e n eventos observados em uma exposição de E horas, a posteriori é $\text{Gamma}(\alpha + n,\ \beta + E)$. A contagem futura na janela w tem a *preditiva* binomial negativa:

$$P(N = j) = \frac{\Gamma(j + a)}{\Gamma(a)\, j!}\, p^a\, (1 - p)^j, \qquad a = \alpha + n, \quad p = \frac{\beta + E}{\beta + E + w}.$$

O sinal da regra é a **cauda** $P(N \geq k) = 1 - \sum_{j<k} P(N = j)$, em que k é o número de tentativas na janela *incluindo o Pix atual* (`negative_binomial_tail`). Os valores adotados: janela de 60 min, retrospecto de 30 dias, prior $\alpha = 1$, $\beta = 24$ (prior de 1 Pix por dia: $\alpha/\beta = 1/24$ por hora), exposição mínima de 24 h; dispara 2FA com cauda < 30 ppm (3×10^{-5}).

Por que serve aqui. Dá um número comparável entre pagadores muito diferentes: quatro Pix numa hora é normal para quem faz um por hora e improvável para quem faz um por semana, sem um corte fixo por pessoa. Com pouco histórico, o prior regulariza a taxa; por isso a regra exige 24 h de exposição, e o pagador novo fica com `VELOCITY_SOURCE_COUNT`.

Verificação de dispersão. A pesquisa recomenda comparar variância e média e trocar por binomial negativa se houver super-dispersão. A análise 2 de `analises.md` mediu, no PixSim, índice de dispersão mediano de 1,01 por hora e 1,03 por dia, com 0,0% (hora) e 2,8% (dia) dos pagadores acima de 1,5, contra 0,0% e 1,2% em contagens Poisson sorteadas com a mesma média: a super-dispersão intra-pagador não passa do ruído amostral. A Poisson–Gamma continua útil porque a *taxa* de cada pagador é incerta, o que alarga a cauda da preditiva.

Limitação. Supõe taxa constante (sem sazonalidade horária ou semanal) e chegadas independentes.

2.4 Distância de haversine e viagem impossível

O que é. A distância ortodrômica entre dois pontos de latitude φ e longitude λ numa esfera de raio $R = 6371$ km é

$$d = 2R \arcsin \sqrt{\sin^2 \frac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2 \frac{\Delta\lambda}{2}}.$$

A velocidade implícita é $v = d / \max(\Delta t_{\text{h}},\ 1/60)$, com piso de um minuto para não dividir por zero quando dois Pix têm o mesmo instante.

Por que serve aqui. Dois Pix com geolocalização a mais de 50 km um do outro e velocidade implícita acima de 900 km/h (um avião comercial) indicam sessão em outro aparelho ou localização falsa.

Limitação. A geolocalização é opcional no contrato; sem ela, a regra não dispara. O PixSim não gera geolocalização, então ela não foi exercitada na validação externa.

2.5 Tipologias de golpe e regras compostas

O que é. Pizzolato et al. [4] propõem uma taxonomia dos golpes de Pix no Brasil (metodologias de ataque, amplificação por IA e estratégias de defesa), descrevendo, entre outros, sequestro relâmpago, acesso remoto à conta da vítima, dispersão do dinheiro em segundos e limites dinâmicos pelo histórico do usuário. Uma *tipologia* é a combinação de sinais que descreve um golpe conhecido.

Por que serve aqui. Sinais isolados (aparelho novo, valor alto, viagem) ocorrem com cliente bom e pedem 2FA; juntos, descrevem tomada de conta e justificam reprovar. Dois 2FA *não* somam uma reprovação: só uma tipologia explícita combina sinais (ADR 0018). Ativas: conta tomada e conta laranja (mula). O fracionamento (`TYPOLOGY_SPLITTING`) foi implementado e descartado pela ablação no PixSim (seção 4.2.7).

Limitação. Cada tipologia é uma hipótese sobre como o golpe se parece; se o simulador ou o fraudador real desenha o golpe de outro jeito, a regra cai a zero (a mula do PixSim é uma conta antiga, não recém-aberta).

2.6 Grafos: funil, dispersão, vizinhança de dois saltos e GraphSAGE

Fonte primária: Hamilton, Ying & Leskovec [18] (GraphSAGE).

O que é. O Pix forma um grafo dirigido pagador $\rightarrow$ destino. A pesquisa descreve dois padrões de conta laranja: *funil* (várias contas pagando quase ao mesmo tempo a uma conta centralizadora) e *dispersão* (a conta recebe e repassa logo em seguida a várias contas). Uma rede neural de grafos (GNN) atribui uma "reputação de rede" a cada conta, calculada fora do caminho síncrono e consultada já pronta.

Reputação por SQL (`graph_sql`). O worker calcula cinco características por conta de destino (pagadores distintos em 24 h n_p; destinos distintos para onde repassou em 6 h n_d; pagadores que também pagaram destinos reprovados, a vizinhança de dois saltos n_h; valor recebido em 24 h v, em centavos; idade da conta idade) e as combina com pesos fixos, sem treino:

$$\text{logit} = -4{,}0 + 1{,}2\ln(1+n_p) + 1{,}5\ln(1+n_d) + 1{,}0\ln(1+n_h) + 0{,}3\ln\!\left(1+\tfrac{v}{100.000}\right) + 1{,}0\cdot\mathbb{1}[\text{idade} < 30], \quad s = \sigma(\text{logit}).$$

O logaritmo amortece caudas pesadas, para que um funil enorme não sature sozinho (`GRAPH_RISK_*` em `src/constants.py`; `src/worker/models/scoring.py`).

GraphSAGE. Para cada nó v e camada ℓ, agrega-se a média dos vizinhos *amostrados* $\mathcal{N}_s(v)$ (no máximo 10, escolhidos por amostragem determinística semeada pelo nó):

$$h_v^{(\ell)} = \text{ReLU}\left(W_{\text{self}}^{(\ell)}\, h_v^{(\ell-1)} + W_{\text{neigh}}^{(\ell)}\, \frac{1}{|\mathcal{N}_s(v)|} \sum_{u \in \mathcal{N}_s(v)} h_u^{(\ell-1)} + b^{(\ell)} \right), \qquad s_v = \sigma(w^\top h_v^{(2)} + b).$$

Duas camadas, dimensão oculta 16, sete características de nó (log do grau de entrada e de saída, log dos valores de entrada e de saída, log da menor idade de conta, fração de pagadores novos, log da dispersão). É indutivo: o mesmo modelo pontua nós que não viu no treino.

Por que serve aqui. A pesquisa insiste que olhar só o pagador é inútil quando a vítima é coagida (os dados do dispositivo são legítimos); a conta de destino é o que denuncia.

Limitação. Inferência de grafo não cabe na latência síncrona, então o score tem atraso de até um ciclo do worker (30 s). O rótulo de nó `is_mule` do PixSim é estático por conta e a janela de avaliação contém os Pix pontuados, então o AUC medido é um teto, não o desempenho do ciclo anterior que o caminho quente usaria.

2.7 Sequências: HMM discreto, Baum–Welch e verossimilhança como anomalia

Fonte primária: Rabiner [19] (HMM e Baum–Welch).

O que é. Um modelo oculto de Markov discreto tem estados latentes $s_t \in \{1, \dots, 3\}$ com transição A, emissão B sobre 12 símbolos e distribuição inicial π. O algoritmo *forward* com fatores de escala c_t calcula

$$\alpha_1 = \pi \odot B_{\cdot,o_1}, \qquad \alpha_t = (\alpha_{t-1} A) \odot B_{\cdot,o_t}, \qquad \log P(o_{1:T}) = \sum_t \log c_t,$$

e o Baum–Welch (EM) estima π, A, B maximizando essa verossimilhança. Cada Pix do pagador vira um símbolo $= 4 \cdot$ faixa $+ 2 \cdot$ destino novo $+$ noite, em que a faixa (baixa, média, alta) compara o valor à mediana do próprio pagador. O score de anomalia é a CDF empírica da log-verossimilhança média negativa dos últimos Pix, em $[0, 1]$ (a `seq_anomaly_likelihood` da pesquisa, que cita Lucas et al. [9]).

Por que serve aqui. Árvores são rápidas, mas cegas à ordem dos eventos; o HMM resume a dinâmica temporal em um número que uma árvore pode consumir.

Limitação. Medido: o payload do Pix traz pouca sequência. No simulador nosso, o AUC cai de 0,652 (com o Pix atual) para 0,509 (só o que o caminho quente vê), praticamente aleatório (ADR 0026). O sinal de sequência só vale com eventos de sessão (login, troca de limite, cadastro de chave) que o enunciado não fornece. Por isso o HMM não foi avaliado no PixSim.

2.8 Regressão logística, calibração de Platt, *gradient boosting* e Isolation Forest

Fontes primárias: Platt [15], Ke et al. [16] (LightGBM), Liu, Ting & Zhou [17] (Isolation Forest).

Regressão logística e Platt. Sobre características padronizadas $\tilde{x}_j = (x_j - \mu_j)/\sigma_j$, a margem é $m = \beta_0 + \sum_j \beta_j \tilde{x}_j$ e a probabilidade calibrada é $p = \sigma(A\,m + B)$, com A, B ajustados em dados de validação (calibração de Platt). A calibração é necessária porque o desbalanceamento (pesos de classe ou subamostragem) desloca as probabilidades: com subamostragem das negativas a uma fração β, a probabilidade corrigida é $p = \beta p_s / (\beta p_s - p_s + 1)$ [6]. O projeto usa `class_weight` balanceado e Platt na validação, não subamostragem.

LightGBM. Soma de árvores de decisão pequenas ajustadas por *gradient boosting*; o score é $\sigma\big(b_0 + \sum_t f_t(x)\big)$, em que cada f_t devolve o valor da folha alcançada. Árvores são exportadas como listas aninhadas `[feature, limiar, esquerda, direita]` e avaliadas em Python puro; a diferença máxima contra a biblioteca em 14.202 Pix foi de $1{,}1 \times 10^{-15}$ (logreg), $1{,}1 \times 10^{-16}$ (LightGBM) e $1{,}7 \times 10^{-16}$ (Isolation Forest).

Isolation Forest. Não supervisionado: árvores aleatórias isolam pontos; anomalias têm caminhos curtos. O score é $s(x, n) = 2^{-\mathbb{E}[h(x)]/c(n)}$, com $c(n) = 2(\ln(n-1) + \gamma) - 2(n-1)/n$ (comprimento médio de caminho numa busca binária sem sucesso) e $\gamma = 0{,}5772\ldots$; $h(x)$ soma a profundidade ao ajuste $c(\text{folha})$ pelo tamanho da folha. Treinado só com legítimos.

Por que servem aqui. Mostram que o desenho comporta modelos aprendidos no caminho síncrono, com inferência de microssegundos e sem inflar a imagem da API com bibliotecas de ML.

Limitações. Dependem de rótulos que não existem em produção; os rótulos sintéticos criam circularidade; o Isolation Forest, treinado só com legítimos do gerador, é sensível à distribuição (seção 6.6).

2.9 Desbalanceamento, custo assimétrico e P@k

O que é. Com fraude rara, acurácia e erro quadrático são inúteis; o certo é otimizar custo [8]. O modelo de custo do projeto (`scripts/backtest/cost-model.json`; *os valores são hipóteses de negócio, não medidas*):

Evento	Custo
Fraude aprovada	valor do Pix (perda direta)
Fraude com 2FA	10% do valor (a vítima coagida às vezes confirma)
Legítimo com 2FA	R$ 2,00 (atrito e abandono)
Legítimo reprovado	R$ 50,00 mais 2% do valor do Pix (cliente bom bloqueado, atendimento, *churn*; mesma definição do PixSim, A3.5, assumida)
Fraude reprovada	zero

(Fonte: ADR 0030, `cost-model.json` e `calibracao.md`, "Custo do falso bloqueio". Os custos medidos antes dessa mudança usavam R$ 50 fixos.)

Como o PixSim superamostra a fraude de propósito, cada Pix de fraude carrega um peso de importância w (0,0071 no mundo A) que desfaz a superamostragem. O custo ponderado por 1.000 Pix é

$$C = 1000 \cdot \frac{\sum_{\text{legít.}} c_i + w \sum_{\text{fraude}} c_j}{N_{\text{legít.}} + w\, N_{\text{fraude}}}.$$

P@k. A capacidade de uma mesa de analistas é fixa; a pergunta é a precisão nos k Pix mais suspeitos do dia, com $k \in \{5, 10, 25, 50\}$ e média diária. A precisão fica inflada pela superamostragem na mesma proporção do peso, e a precisão *ponderada* dos alertas, com a taxa real de fraude, é da ordem de 1%.

Limitação. O custo absoluto depende das premissas do modelo de custo; o que se compara é a diferença entre configurações.

2.10 PSI e deriva; rótulos atrasados; viés de seleção; *champion/challenger* e *shadow mode*

PSI (*Population Stability Index*). Compara a distribuição de uma característica entre duas janelas, em B intervalos (aqui 10 por quantil da amostra-base, com piso de 0,0001 por intervalo): PSI $= \sum_{i=1}^{B} (q_i - p_i) \ln(q_i/p_i)$. Leitura convencional: abaixo de 0,1 estável; 0,1 a 0,25 atenção; acima de 0,25 deriva. Serve de alarme para recalibrar ou retreinar (a pesquisa o liga a *concept drift* em Dal Pozzolo et al.).

Rótulos atrasados. Dal Pozzolo et al. [7] tratam do caso em que a fraude só é confirmada depois: exemplos recentes ainda não rotulados entram no treino como legítimos, o que enviesa o modelo contra o padrão mais novo. No Pix, o MED é a fonte desse atraso.

Viés de seleção. Um Pix que o motor reprova não liquida e portanto não gera desfecho; o que a instituição observa depois é censurado pelo próprio motor. Quanto melhor o motor, menos desfechos ele enxerga. É o problema clássico da inferência de rejeitados.

Champion/challenger e shadow mode. O *campeão* decide; o *desafiante* roda em paralelo, sem decidir, e é promovido se custar menos na janela de comparação. Em *shadow mode* um modelo ou regra candidata calcula e grava o que decidiria, sem alterar a resposta; isso prova que o modelo cabe no caminho síncrono (latência, dependências, serialização) sem expor o cliente a decisões guiadas por rótulo sintético (ADR 0022).

Limitação. Em janelas curtas e com poucos retreinos, a comparação campeão–desafiante é ruidosa (seção 6.8).

3 Arquitetura

3.1 Camadas do Base Service

O projeto parte do `bootcamp-base-api` (ADR 0001), um serviço FastAPI em camadas em que a dependência anda num sentido só: `resources` → `controllers` → (`repositories` | `rules`). Cada pasta tem um papel e uma proibição:

Camada	Faz	Não faz
Middlewares	identificador da requisição, log, `INTERNAL-TOKEN` (403), sessão de banco	conhecer uma rota
Recursos (HTTP)	validar o corpo pelo JSON Schema (400), chamar o controller, escolher 201 ou 200	regra, `raise`
Controllers	idempotência, montagem do contexto, execução das regras, decisão, gravação; único `commit`, como última linha antes do `return`	SQL direto
Regras	decidir se *uma* regra dispara, a partir do Pix, do contexto e dos parâmetros	banco, ambiente, relógio, ler outra regra
Repositórios	consultas e agregados por janela	decidir se é fraude
Schemas	JSON Schema com `additionalProperties: false`	regra
DTOs	montar `{key, status, metadata}`	serializar `id` interno ou chave estrangeira
Worker	jobs periódicos do *cold path*, cada um com o seu `commit`	ser chamado pela API; decidir um Pix

Seis regras de projeto não se negociam: teste nunca importa de `src/` nem lê o banco (R1, *black box* por HTTP); um comando sobe tudo numa máquina limpa (R2, `make all`); decisão `rejected` não é erro, sai com status 201 (R3, ADR 0003); tabelas são *append-only* e todo Pix recebido é gravado, inclusive os reprovados (R4); o `id` interno do banco nunca sai, a saída tem `key` UUID (R5); dinheiro é `BIGINT` em centavos, nunca ponto flutuante (R6).

3.2 *Hot path* × *cold path*

A pesquisa separa o que cabe no caminho síncrono do que não cabe. O projeto implementa as duas metades:

	Hot path (dentro do `POST`)	*Cold path* (container `worker`)
Executa	15 regras que decidem; 5 modelos em *shadow*	reputação de grafo por SQL, GraphSAGE e HMM
Cadência	por requisição	a cada 30 s (`WORKER_INTERVAL_SECONDS`)
Orçamento de tempo	milissegundos	segundos
Entrega ao *hot path*		scores gravados em `account_risk_score` e `payer_sequence_score` (*append-only*)
Se o score falta ou está velho (> 120 min)		a regra de grafo fica muda; nada quebra

O *hot path* só **lê** o score pronto. O relógio do worker é o maior `transaction_date` da base, coerente com o relógio das regras (ADR 0007), e um ciclo é pulado se nada mudou. Os modelos aprendidos do worker dependem de `numpy`, que só existe no alvo `worker` do Dockerfile; a imagem da API não ganhou nenhuma biblioteca de ML (ADR 0025 e 0026).

3.3 Diagramas C4

Os diagramas abaixo (PlantUML em `docs/02-arquitetura/diagramas/`) mostram o sistema em três níveis, mais a visão offline.

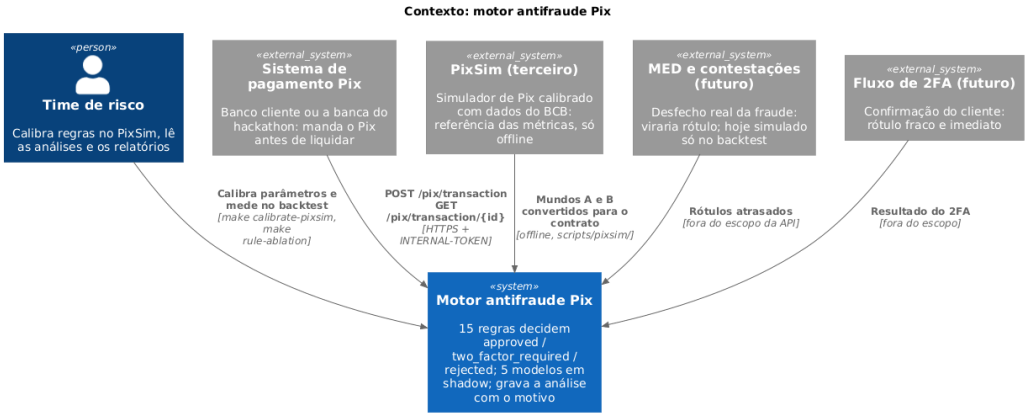


Figura 1: Contexto: quem fala com o motor (C4, nível 1).

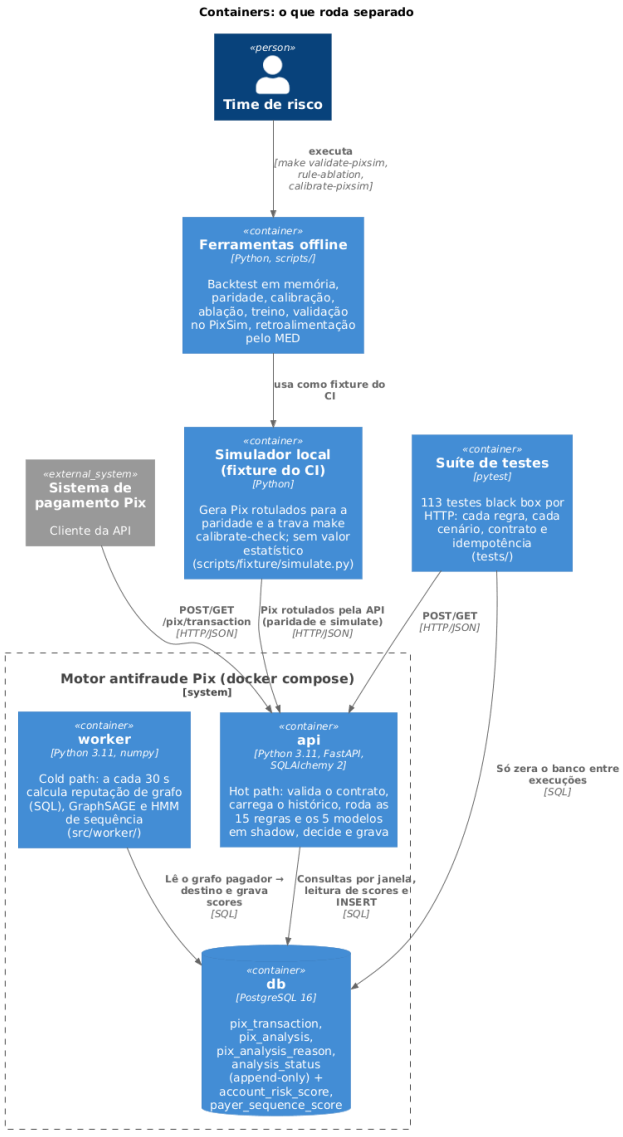


Figura 2: Containers: o que roda separado (C4, nível 2).

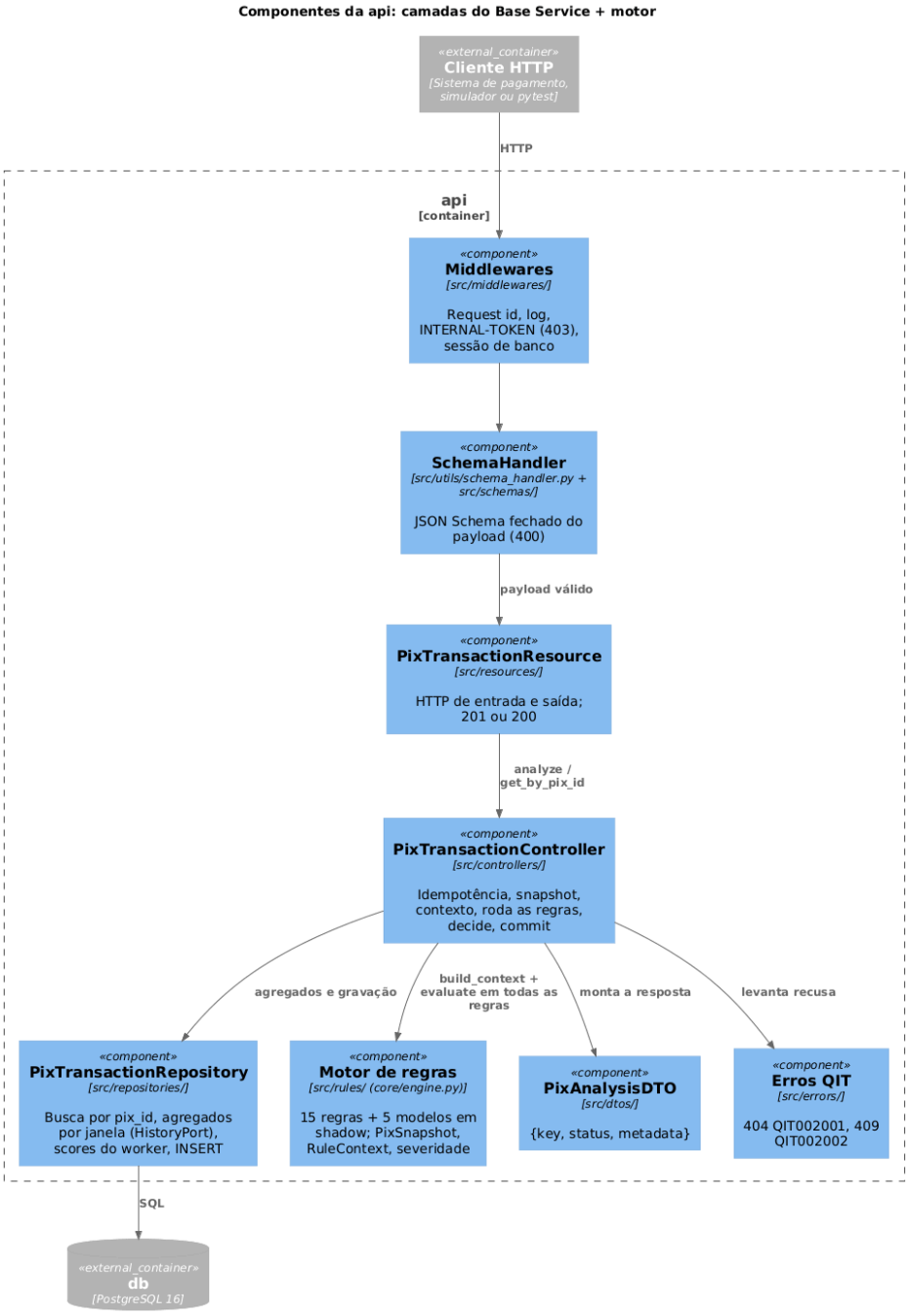


Figura 3: Componentes da API: camadas do Base Service mais o motor (C4, nível 3).

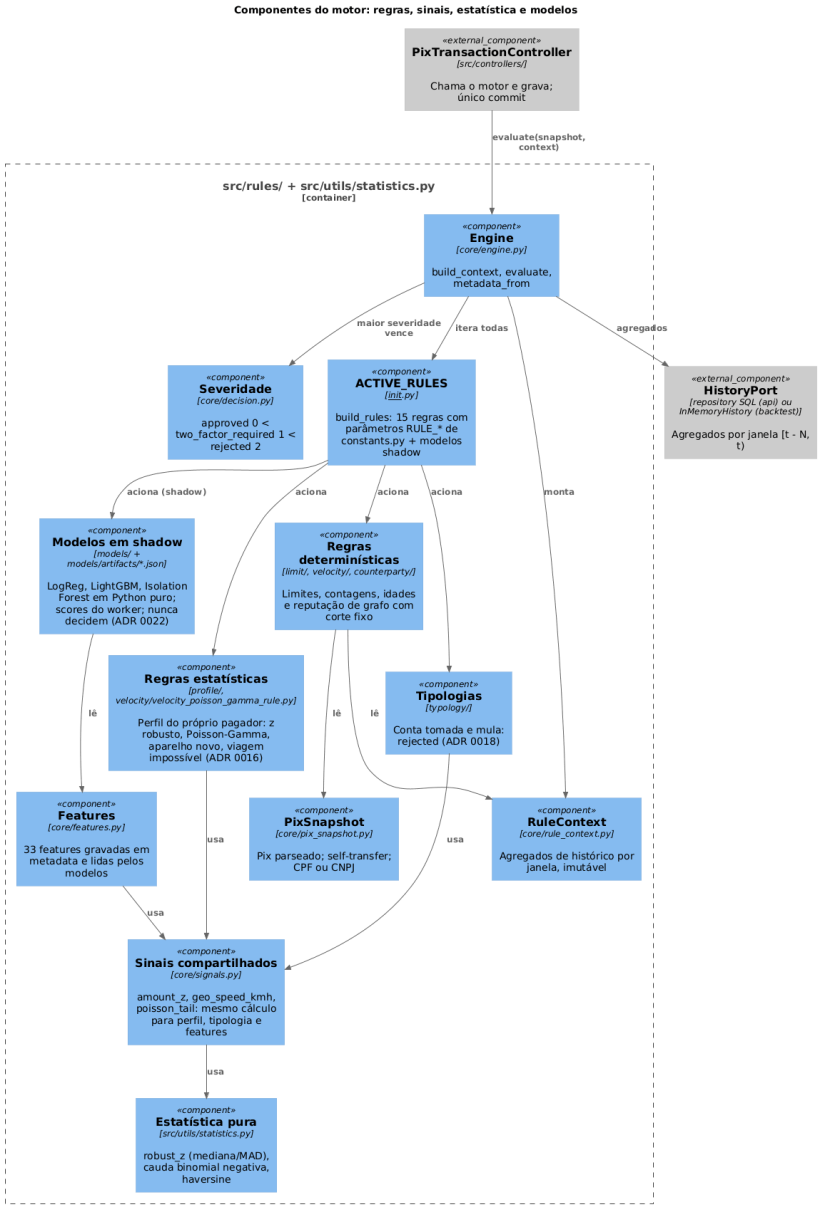


Figura 4: Componentes do motor: regras, sinais, estatística e modelos; o mesmo código em produção e no backtest (C4, nível 3).

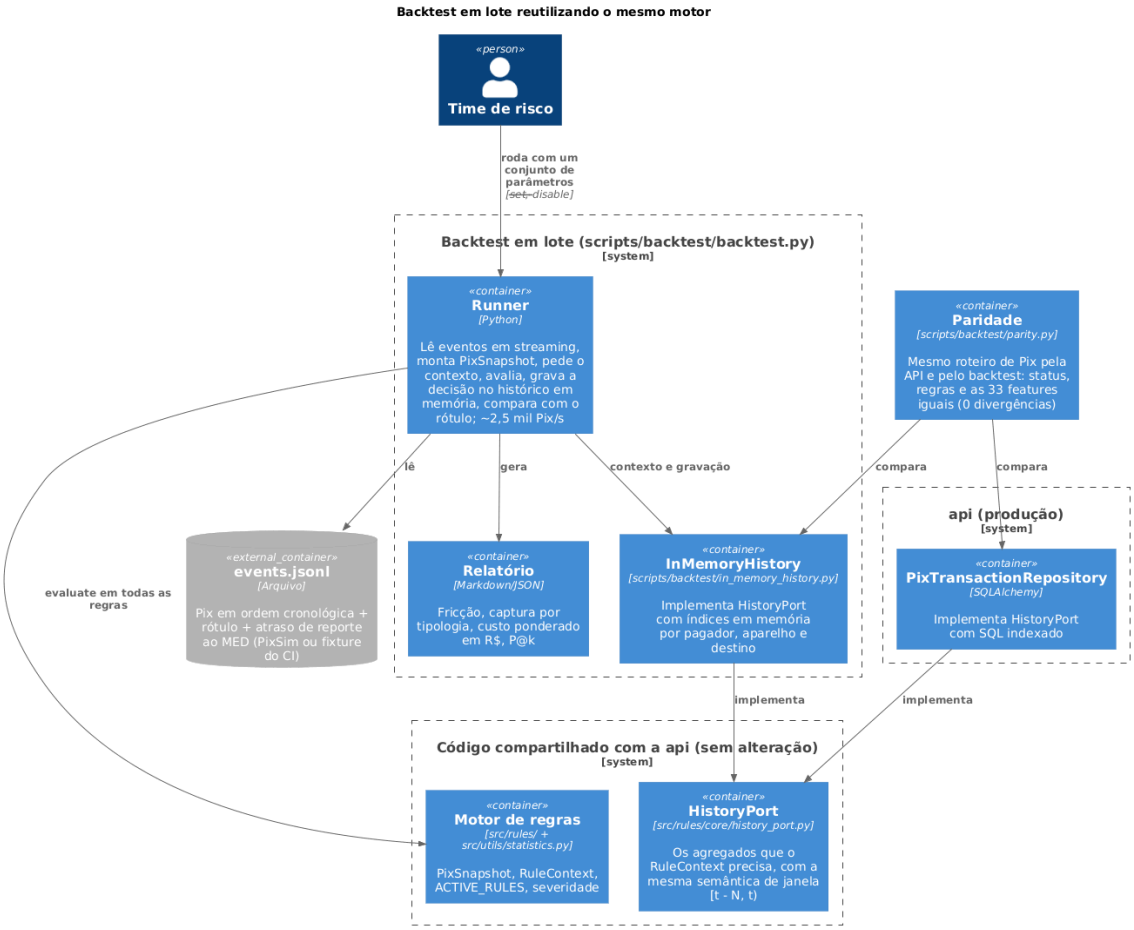


Figura 5: Backtest em lote e treino, que reutilizam o mesmo motor (visão offline).

3.4 Contrato da API

Rota	Faz	Sucesso	Recusas
`POST /pix/transaction`	analisa e grava um Pix	201 `{key, status, metadata}`; 200 se o `id` já foi analisado com o mesmo corpo	400 `QIT000001` (corpo torto); 409 `QIT002002` (mesmo `id`, corpo diferente)
`GET /pix/transaction/{id}`	devolve a análise gravada, sem recalcular	200	404 `QIT002001`
`GET /`, `GET /health_check`	identificação e saúde	200, 204	

Toda rota, exceto `/` e `/health_check`, exige o cabeçalho `INTERNAL-TOKEN` (403 `QIT000002`). A saída é exatamente `{key, status, metadata}`. O `metadata` (livre no enunciado) traz:

Campo	Conteúdo
`reasons`	um item por regra disparada (`rule`, `decision`, `reason` em português), do mais severo ao menos severo; vazia em `approved`
`signals`	por regra disparada, os números que ela usou (idade da conta, contagem, z, km/h, …)
`features`	vetor de características do Pix (33 campos, nada identificável), fonte das características de treino
`shadow`	o que cada regra ou modelo em sombra *decidiria* (`rule`, `would_decide`, `reason`, `signals`)
`rule_set_version`	versão do conjunto de regras (ADR 0021), hoje `2026.10.18-pixsim-2`

A idempotência usa o `id` do Pix como chave natural, com `UNIQUE` no banco; a corrida entre dois `POST` iguais é resolvida pelo banco (ADR 0012). Os schemas de entrada ficam em JSON Schema (ADR 0013).

3.5 Modelo de dados

Quatro tabelas do histórico, todas *append-only* (R4), mais duas do *cold path*:

Tabela	Papel	Detalhes
`analysis_status`	`approved` (0), `two_factor_required` (1), `rejected` (2)	a coluna `severity` faz de "o mais severo vence" um `ORDER BY`
`pix_transaction`	o fato: o Pix recebido	colunas tipadas (valor em `BIGINT` centavos, `TIMESTAMPTZ` com fuso, `INET`) mais `payload JSONB` para reprocessar e comparar corpo na idempotência; `pix_id UNIQUE`
`pix_analysis`	a opinião: decisão, `metadata`, `rule_set_version`	`analysis_key` é a `key` pública; separada do fato porque têm ciclos diferentes
`pix_analysis_reason`	uma linha por regra disparada	calibrar uma regra é contar disparos com SQL, sem abrir JSON
`account_risk_score`	score de reputação por conta (`graph_sql`, `graphsage`)	gravado pelo worker, com as contribuições de cada característica
`payer_sequence_score`	score de anomalia de sequência (HMM)	gravado pelo worker

Índices por janela servem cada agregado como *range scan*: `(documento do pagador, data)`, `(aparelho, data)`, `(documento do destino, data)` e `(pagador, destino)`. Em medição (300 mil Pix em 30 dias), a consulta de pares do descartado grupo de pares levou 8,7 ms com o índice por data e 371 ms sem ele (*fonte: `validacao-pixsim.md`, "Peer group"*), o que confirma a escolha de indexar toda janela.

4 Implementação

4.1 O motor: *snapshot*, contexto, `HistoryPort` e decisão

O motor foi extraído do controller para poder rodar fora da API sem criar uma segunda versão das regras (ADR 0023). As peças, por responsabilidade:

Peça	Responsabilidade
`PixSnapshot`	o Pix já interpretado (datas com fuso, `is_self_transfer`, `is_legal_entity_source`)
`RuleContext`	dataclass imutável com os agregados de histórico relativos a t = `transaction_date`
`HistoryPort`	interface (Protocol) com os agregados de que o contexto precisa; cada método exclui o próprio Pix e olha só antes de t
`build_context(snapshot, history)`	monta o contexto chamando o `HistoryPort`
`evaluate(snapshot, context, rules)`	roda *todas* as regras, ordena os disparos por severidade e devolve a decisão e o vetor de características
`metadata_from(resultado)`	monta `reasons`, `signals`, `features`, `shadow` e `rule_set_version`
Regra (`BaseRule`)	`code`, `family`, `evaluate(pix, context) -> RuleResult | None`; pura: sem banco, ambiente nem relógio

Dois adaptadores implementam o `HistoryPort`: o repositório SQL, na API, e um histórico em memória, no backtest. O agregado novo entra no *port*, nos dois adaptadores e no vetor de características, e a paridade acusa se faltar um lado.

Janelas. São $[t-N, t)$, com t o `transaction_date` do próprio Pix, nunca `datetime.now()` (ADR 0007): é o que torna a decisão reprodutível e o backtest idêntico à API. Tentativas contam todos os status; somas e "conhecido" ignoram os reprovados, porque o dinheiro que não saiu não é exposição (ADR 0021).

Decisão por severidade. Todas as regras rodam, sem curto-circuito; cada uma devolve nada, `two_factor_required` ou `rejected`; a decisão final é a de maior severidade, e sem disparo é `approved` (ADR 0006). A ordem das regras não muda a decisão, só a ordem dos motivos (severidade e depois a ordem de `ACTIVE_RULES`). O contexto é montado uma única vez por Pix; no `POST`, o controller faz idempotência, chama o motor, grava Pix, análise e motivos e dá o `commit` na última linha.

Parâmetros. Todo parâmetro de regra é uma constante `RULE_*` em `src/constants.py`, lida do ambiente (ADR 0008); não há número mágico em regra. Mudar um parâmetro exige evidência do pipeline de calibração, registro do porquê e novo `RULE_SET_VERSION`.

Regras em sombra. `BaseRule.shadow = True` faz o motor rodar a regra e mandar o resultado só para `metadata.shadow`; nunca afeta `status`, `reasons` nem `pix_analysis_reason` (ADR 0022). Promover é trocar um atributo, registrar e mudar a versão.

4.2 Catálogo de regras, parâmetros e justificativas

> **Nota.** O motor tem **15 regras que decidem**, na versão `2026.10.18-pixsim-2` de `src/constants.py`. A coluna "Justificativa" resume `docs/05-motor/regras.md`, que separa a fonte regulatória (BCB, conferida no normativo), a literatura e a evidência medida no PixSim; o que não foi verificado está marcado. A contribuição medida de cada regra no PixSim está na seção 6.12 (ablação). Os parâmetros marcados com ($\star$) foram ajustados no PixSim pelo custo ponderado (seção 6.5).

As janelas são relativas ao instante da transação. "R$" abaixo é valor em reais; no código são centavos inteiros.

4.2.1 Limite: "passou do teto?"

Regra	Lógica	Parâmetros	Decisão	Justificativa
LIMIT_NIGHT_AMOUNT	Pix entre 20h e 6h (hora local do payload) acima do teto; não vale para CNPJ	R$ 1.000	2FA	O BCB fixou R$ 1.000 como limite noturno padrão de pessoa física (Res. BCB 142/2021, hoje IN BCB 512/2024) [11]; o art. 39-B, §1º, III do Regulamento do Pix [12] cita horário e dia. Mantido por âncora regulatória, embora o PixSim prefira um corte maior. 2FA e não reprovação porque o cliente pode ter ampliado o limite
LIMIT_SINGLE_AMOUNT	Valor de um Pix acima do teto absoluto	R$ 50.000 (CPF), R$ 200.000 (CNPJ)	2FA	Teto de exposição da instituição; o art. 37 do Regulamento do Pix [12] só permite limite por critério de risco e perfil, sem fixar valor. Valor do time, **não verificado** em fonte externa

4.2.2 Velocity: "rápido demais?"

Regra	Lógica	Parâmetros	Decisão	Justificativa
VELOCITY_SOURCE_COUNT	Tentativas do pagador em 10 min, contando as reprovadas	2FA ≥ 5; reprova ≥ 10	2FA / reprova	Conta tomada é esvaziada em rajada: transferências fraudulentas são executadas em segundos (Pizzolato et al. [4]); contar também as reprovadas é escolha do time (ADR 0021). Protege o pagador sem histórico. Limites do time, sem fonte externa
VELOCITY_DEVICE_SOURCES	Pagadores distintos no mesmo aparelho em 24 h	reprova > 3	reprova	Fazenda de contas ou golpista com credenciais de vítimas; "família no mesmo celular raramente passa de 3" é premissa do time, **não verificada**
VELOCITY_POISSON_GAMMA	Cauda da preditiva Poisson–Gamma da rajada da última hora (seção 2.3), com ≥ 24 h de exposição	($\star$) cauda < 30 ppm; prior $\alpha = 1$, $\beta = 24$ h	2FA	O "rápido demais" de cada um: conjugada Gamma–Poisson aplicada à comparação com o próprio histórico de Bolton & Hand [3]. Corte escolhido no PixSim

4.2.3 Montante: "somou demais?"

Nenhuma regra ativa. A única regra da família, AMOUNT_SOURCE_DAILY_SUM (soma do pagador em 24 h), foi descartada pela ablação (seção 4.2.7). O teto por Pix (LIMIT_SINGLE_AMOUNT) e o z robusto (PROFILE_AMOUNT_ROBUST_Z) cobrem parte da pergunta. A soma em 24 h continua no vetor de características, porque os modelos em *shadow* a leem.

4.2.4 Perfil: "está no padrão do cliente?"

Regra	Lógica	Parâmetros	Decisão	Justificativa
PROFILE_NEW_DEVICE	Pagador com histórico, aparelho nunca visto, valor alto	($\star$) a partir de R$ 200	2FA	Troca de aparelho é o primeiro passo da credencial comprometida e do acesso remoto (Pizzolato et al. [4]); desvio do comportamento do próprio titular (Bolton & Hand [3]). O piso evita fricção no cafezinho

Regra	Lógica	Parâmetros	Decisão	Justificativa
PROFILE_NEW_SOURCE_ACCOUNT	Conta de origem com menos de 7 dias e valor alto	< 7 dias e $>$ R$ 2.000	2FA	Conta aberta com identidade falsa costuma ser usada logo; o art. 39-B do Regulamento do Pix [12] cita a idade da conta, mas trata do PSP do *recebedor*; o uso para a conta de origem é analogia
PROFILE_AMOUNT_ROBUST_Z	z robusto de $\ln(\text{valor})$ sobre 30 dias do pagador (seção 2.2), ≥ 5 Pix	($\star$) $z > 4{,}5$; piso de escala 25%	2FA	Escore z modificado com mediana e MAD (Iglewicz & Hoaglin [14], corte de 3,5 sob normalidade) sobre o perfil do titular (Bolton & Hand [3]); 4,5 em vez de 3,5 por calibração no PixSim
PROFILE_GEO_VELOCITY	Distância ao último Pix com GPS dividida pelo tempo decorrido (seção 2.4)	> 50 km e > 900 km/h	2FA	Viagem impossível: sessão em outro aparelho ou localização falsa; 900 km/h (avião comercial) é convenção do time, **não verificada**

4.2.5 Contraparte: "quem recebe é confiável?"

Regra	Lógica	Parâmetros	Decisão	Justificativa
COUNTERPARTY_NEW_ACCOUNT	Idade da conta de destino; autotransferência não conta	2FA < 30 dias; reprova < 3 dias e $>$ R$ 1.000	2FA / reprova	Contas laranja são abertas para receber o golpe e esvaziadas em dias; com vítima coagida só o destino denuncia. O art. 39-B, §1º, II do Regulamento do Pix [12] torna o tempo desde a abertura da conta do recebedor fator obrigatório da avaliação; os cortes são do time
COUNTERPARTY_FAN_IN	Pagadores distintos para o mesmo destino em 1 h	reprova > 10	reprova	Conta de passagem recebendo de muitas vítimas ao mesmo tempo (funil, da pesquisa [2]); limiar do time, sem fonte externa
COUNTERPARTY_FIRST_TIME_HIGH	Primeiro Pix (não reprovado) do pagador para esse destino, valor alto	($\star$) $>$ R$ 1.000	2FA	Falso parente e falso boleto vão para destino novo; o aluguel vai sempre ao mesmo. O art. 39-B, §1º, IV do Regulamento do Pix [12] cita a recorrência entre pagador e recebedor. É a regra de maior efeito no PixSim
COUNTERPARTY_GRAPH_RISK	Score de reputação do destino calculado pelo worker (seção 2.6), com até 120 min	2FA $\geq 0{,}6$; reprova $\geq 0{,}9$	2FA / reprova	Conta laranja é padrão de rede; o motivo cita as duas características que mais pesaram. Cortes do time

4.2.6 Tipologia: combinações que descrevem um golpe

Regra	Lógica	Parâmetros	Decisão	Justificativa
TYPOLOGY_ACCOUNT_TAKEOVER	Pagador com histórico, aparelho novo e (viagem impossível ou $z > 3{,}0$)	$z > 3{,}0$	reprova	Tomada de conta (Pizzolato et al. [4]): cada sinal sozinho pede 2FA; juntos o risco é claro (ADR 0018)
TYPOLOGY_MULE	Destino com < 30 dias e ≥ 5 pagadores distintos em 1 h	5 pagadores	reprova	Conta laranja: combina a idade da conta do recebedor (art. 39-B, §1º, II [12]) com o funil; barra no 5.º pagador, antes do fan-in genérico (11.º)

4.2.7 Regras descartadas

Três regras saíram do motor na versão `2026.10.18-pixsim-2` (ADR 0033). O critério: uma regra fica se tiver valor por pelo menos um caminho: (1) **contribui no PixSim** (ablação positiva); (2) tem **âncora regulatória verificada** no BCB, mesmo com o PixSim preferindo outra coisa; ou (3) **cobre um sinal que o PixSim não gera** (geolocalização, rajada, fazenda de aparelhos, funil, grafo do worker): não dispara lá, logo custa zero ali, e cobre um golpe que o gerador não tem. Sai a regra com ablação negativa e nenhum dos três caminhos.

Regra descartada	Família	Ablação no mundo B (R$ por 1.000 Pix)	Por que sai
`TYPOLOGY_SPLITTING`	tipologia	$-$25,99	Sem fonte regulatória nem acadêmica para os números; o PixSim não tem fracionamento, então a regra só reprovava legítimos
`COUNTERPARTY_FLAGGED`	contraparte	$-$11,76	Só análoga ao art. 39-B, §1º, I (usa as nossas reprovações, não notificações de infração do DICT); 2FA em 0,72% dos legítimos com *lift* de 1,8x
`AMOUNT_SOURCE_DAILY_SUM`	montante	$-$8,69	Nenhum normativo conferido fixa soma diária de Pix; custo em platô entre R$ 30 mil e R$ 60 mil

(Ablação com as 18 regras, mundo B, falso bloqueio a R$ 50 + 2%.) Para não transformar o mundo B em mundo de ajuste, o descarte foi confirmado no mundo A, que escolhe parâmetros:

Mundo	Custo com 18 regras	Custo com 15 regras	Pix de vítima barrados	2FA em legítimos	Reprovação em legítimos
A (escolhe)	R$ 132,13	R$ 109,61	51,7% → 50,7%	3,81% → 2,97%	0,012% → 0,000%
B (confirma)	R$ 153,51	R$ 118,82	51,8% → 51,1%	4,13% → 3,09%	0,03% → 0,002%

Cada uma, desligada sozinha no mundo A, também baixa o custo (`TYPOLOGY_SPLITTING` R$ 115,69; `COUNTERPARTY_FLAGGED` R$ 122,27; `AMOUNT_SOURCE_DAILY_SUM` R$ 125,93). A trava do CI no nosso simulador continua passando (6,6% de 2FA em legítimos, nenhuma reprovação, 100% das sequências de fraude barradas; o fracionamento passa a ser barrado por 2FA em vez de reprovação). Ficam, mesmo com ablação negativa: `COUNTERPARTY_NEW_ACCOUNT` e `TYPOLOGY_MULE` (art. 39-B, §1º, II), `LIMIT_NIGHT_AMOUNT` (Res. BCB 142/2021) e `PROFILE_AMOUNT_ROBUST_Z` (Iglewicz & Hoaglin [14]). As características que as três liam continuam no contexto, porque os modelos em *shadow* as usam; o código e os testes delas estão no histórico do Git até o commit `0bb5df8`.

Ideias que não viraram regra. Horário atípico por conta (a pesquisa indica que golpes de engenharia social acontecem em horário comercial, então um corte fixo não ajuda); distância entre `transaction_date` e o recebimento (depende de saber a hora real da chegada); IP compartilhado por muitos pagadores (sinal fraco, por NAT de operadora móvel); grupo de pares (`PROFILE_PEER_AMOUNT`, implementado, medido e descartado; seção 6.9).

(Fonte: `docs/05-motor/regras.md`, ADR 0033, `validacao-pixsim.md`.)

4.3 Modelos servidos em Python puro

Treino fora da API (`scikit-learn`, `lightgbm` e `numpy` só em dev); exportação para JSON versionado; inferência em `math` puro dentro da API (ADR 0024):

Modelo	Treino	Arquivo	Inferência
Regressão logística + Platt	`class_weight` balanceado	pesos, médias, desvios, A e B de Platt	margem padronizada $\rightarrow$ $\sigma(Am + B)$
LightGBM	100 árvores, 15 folhas, `scale_pos_weight`	árvores como listas aninhadas, `base_score`	soma das folhas $\rightarrow \sigma$
Isolation Forest	só legítimos	árvores, tamanho das folhas, `n_samples`	comprimento médio de caminho

Os três são regras em sombra (`MODEL_LOGREG`, `MODEL_LIGHTGBM`, `MODEL_IFOREST`); os arquivos são lidos uma vez, e se um faltar a regra não é registrada e a API sobe. Verificou-se (`make model-parity`) a equivalência com as bibliotecas sobre 14.202 Pix: diferença máxima de $1{,}1 \times 10^{-15}$. O custo de latência dos três modelos juntos foi de 149 μs por Pix no backtest (ADR 0024, medido no simulador nosso). Cada arquivo carrega `trained_on` com a origem (SHA do PixSim e semente).

4.4 O worker (*cold path*)

O container `worker` usa a mesma imagem de base e o mesmo código da API (`python -m worker`) e roda três jobs a cada ciclo, sobre os destinos que receberam Pix nas últimas 72 h:

1. **Reputação de grafo (`graph_sql`).** Características em SQL (funil, dispersão, dois saltos, valor, idade da conta) e o escore logístico de pesos fixos da seção 2.6; grava score e contribuições.
2. **GraphSAGE.** Monta o subgrafo da janela por SQL, amostra até 10 vizinhos por nó, executa o *forward* em `numpy` e grava o score.
3. **HMM.** Reconstrói a sequência de símbolos de cada pagador (30 dias de histórico), calcula a anomalia pela verossimilhança e grava o score.

Ciclo medido no simulador nosso: 0,31 s para os três jobs (619 destinos, 179 pagadores); a imagem do worker tem 546 MB, contra 404 MB da API. Os testes de integração do worker rodam um ciclo explícito (`docker compose run --rm worker python -m worker --once`), com o worker parado durante a suíte, como processo externo, sem importar `src/`.

4.5 Backtest em lote e paridade

O backtest roda o **mesmo** motor sobre um arquivo de eventos, sem HTTP e sem banco, usando o histórico em memória: 2.506 Pix/s sobre os 254.795 Pix do PixSim (101,7 s). Como o histórico em memória é uma segunda implementação dos agregados, existe o risco de *training–serving skew*; a defesa é a **paridade**: o mesmo arquivo de eventos passa pela API e pelo backtest, e status, regras disparadas e o vetor inteiro de características precisam ser idênticos Pix a Pix (`make parity`). Como o vetor cobre todos os agregados, a igualdade prova que os dois adaptadores contam igual sem que nenhum teste importe `src/` (R1). No PixSim a paridade foi verificada em 500 Pix (cerca de 40 de fraude, janela na virada do burn-in): **0 divergências** (`make pixsim-parity`; *fonte: `validacao-pixsim.md`, "Limitações"*). Fora dessa amostra o backtest em memória não foi comparado com a API.

No backtest os scores do *cold path* são `None` (o histórico em memória não executa o worker), o que mantém a paridade exata e significa que as regras de grafo, GraphSAGE e HMM não foram exercitadas na validação do PixSim em lote.

4.6 Mapa do código

Esta seção concentra os caminhos de arquivo; o restante do relatório descreve módulos por responsabilidade. Atualize apenas esta tabela se a estrutura mudar (a mesma tabela, mais detalhada, está em `docs/02-arquitetura/camadas.md`, "Onde fica o quê", e em `scripts/README.md`).

Responsabilidade	Onde
Aplicação, middlewares, erros	`src/app.py`, `src/middlewares/`, `src/errors/`
Rota HTTP, schema, DTO	`src/resources/pix_transaction.py`, `src/schemas/post_pix_transaction.json`, `src/dtos/`
Controller (idempotência, gravação, `commit`)	`src/controllers/pix_transaction_controller.py`
Consultas e agregados SQL	`src/repositories/` (Pix, grafo, GraphSAGE, sequência)
Tabelas	`database/database.sql`, `src/models/`
Parâmetros `RULE_*` e `RULE_SET_VERSION`	`src/constants.py`
Lista ativa (`build_rules`, `ACTIVE_RULES`)	`src/rules/__init__.py`; a ordem da lista desempata motivos de mesma severidade
Motor (`engine.py`), `BaseRule`, decisão, `PixSnapshot`, `RuleContext`, `HistoryPort`, características, sinais compartilhados	`src/rules/core/`
Regras (uma classe por arquivo, `<familia>_<sinal>_rule.py`)	`src/rules/limit/`, `velocity/`, `profile/`, `counterparty/`, `typology/`
Estatística pura (z robusto, cauda Poisson–Gamma, haversine)	`src/utils/statistics.py` (fica em `utils/` porque os scripts de análise também a usam)
Modelos servidos em Python puro	`src/rules/models/` (`logreg`, `gbdt`, `iforest`, regras em *shadow*) e `src/rules/models/artifacts/*.json`
Cold path	`src/worker/__main__.py`; jobs em `src/worker/jobs/` (grafo, GraphSAGE, HMM); modelos em `src/worker/models/` (`scoring.py`, `graphsage.py`, `hmm.py`, `artifacts/*.json`)
Nosso simulador (fixture de CI)	`scripts/fixture/` (`simulate.py`, `replay.py`, `report.py`, `api_client.py`)
Backtest, histórico em memória, paridade, custo	`scripts/backtest/` (`backtest.py`, `in_memory_history.py`, `parity.py`, `costs.py`, `cost-model.json`)
Calibração, trava do CI, ablação	`scripts/calibration/` (`calibrate.py`, `calibration-grid-pixsim.json`, `calibration-gate.json`, `rule_ablation.py`)
PixSim	`scripts/pixsim/` (`pixsim_setup.sh`, `pixsim_export.py`, `pixsim_adapter.py`, `pixsim_report.py`, `pixsim_scores.py`, `pixsim_waterfall.py`)
Treino e avaliação de modelos, rótulos atrasados, ruído do escorer	`scripts/models/` (`train_models.py`, `train_graphsage.py`, `train_hmm.py`, `export_features.py`, `model_eval.py`, `check_model_parity.py`, `delayed_labels.py`, `scorer_noise.py`)
Retroalimentação pelo MED; análises estatísticas	`scripts/feedback/` (`feedback.py`, `feedback_core.py`); `scripts/analysis/analyze.py`
Testes *black box* (113, só por HTTP)	`tests/`
Orquestração	`Makefile`, `docker-compose.yml`, `Dockerfile`, `.github/workflows/ci.yml`
Documentação e decisões	`docs/` (ADRs em `docs/02-arquitetura/decisoes/`)

Os scripts se importam pelo nome do módulo: cada um começa importando `scripts/_paths.py`, que põe `src/` e as pastas de `scripts/` no `sys.path`; toda saída vai para `scripts/out/`, fora do Git.

5 Metodologia de validação

5.1 Por que o nosso simulador é só *fixture* de CI

O primeiro simulador (`scripts/fixture/simulate.py`, ADR 0020) gera pagadores legítimos e injeta cinco tipologias de fraude que o próprio time desenhou (conta tomada, mula, fracionamento, golpe noturno e fazenda de contas). As regras foram escritas procurando exatamente esses padrões, logo a medição é **circular**: sobre ele, a captura de Pix de vítima chegava a 98,5% e metade do valor das vítimas era reprovada, enquanto no PixSim o mesmo motor barrou cerca de 36% dos Pix de vítima antes da calibração (ADR 0029; *fonte:* `validacao-pixsim.md`*, "O que se mantém e o que cai"*). Os modelos treinados nele também não generalizaram: o LightGBM obteve AUC 0,58 e a regressão logística 0,77 no PixSim (ADR 0031).

Decisão (ADR 0029): todo número de resultado vem do PixSim. O nosso simulador permanece porque é rápido, determinístico e roda em máquina limpa, e serve a duas coisas: os testes de paridade e a **trava de regressão** `make calibrate-check`, que roda o backtest com os padrões sobre o simulador (semente 42, 200 pagadores, 14 dias) e falha se passar de legítimos reprovados $\leq 0{,}1\%$, 2FA em legítimos $\leq 7{,}0\%$ ou menos de 100% das sequências de fraude com algum Pix barrado. A trava não é meta e não justifica nenhum parâmetro. Medido com as 15 regras: nenhuma reprovação, 6,6% de 2FA, 100% das sequências (*fonte: ADR 0033*).

5.2 O PixSim

O PixSim [5] é um simulador aberto (Apache-2.0) de fraude em pagamentos instantâneos, calibrado com dados abertos do Banco Central. Gera contas (pessoa física e jurídica, região, faixa etária, idade da conta), tráfego legítimo (Poisson não homogêneo por conta), quatro tipologias de fraude do lado do pagador (engenharia social, conta tomada, Pix errado e QR adulterado) e cadeias de dispersão por mulas até contas de saída, com atraso de reporte ao MED. Usamos o commit `7fdfc90013960f46df60c3ea01838e768e7a420a`. Foi usada **apenas a geração de transações**: a política de analistas, bloqueios e recuperação pelo MED do artigo não foi usada.

Código de terceiro, tratado como não confiável. Antes de executar foram lidos o README, as dependências e os módulos chamados; não rodamos os testes, notebooks nem instaladores dele. As dependências pinadas foram instaladas em venv próprio (Python 3.13), sem privilégio, e o gerador roda com `python -I` e limite de memória. Tudo fica em `scripts/out/pixsim/`, fora do Git.

5.3 Escala reduzida

Item	Valor
Contas	5.025 (5.000 mais 25 contas de saída do PixSim)
Janela avaliada	30 dias (mundo A: semente 20260910)
Burn-in	30 dias antes do dia 0, 122.967 Pix legítimos enviados ao motor como histórico e excluídos das métricas
Pix totais	254.795 (125.518 legítimos avaliados)
Eventos de fraude	986: 618 engenharia social, 209 conta tomada, 91 Pix errado, 68 QR adulterado
Pix de fraude	6.310: 1.380 pagador→mula, 91 sementes de Pix errado, 1.771 `mule_forward`, 3.068 `cash_out`
Peso de importância da fraude	0,0071

(Fonte: `validacao-pixsim.md`*, "Escala usada" e "Tamanho avaliado".)* O artigo roda 1,08 milhão de contas (19 GB por parte); a máquina de desenvolvimento tem cerca de 2 GB livres. O backtest lê os eventos em *streaming* (pico de memória de 2,07 GB para 0,52 GB, saída idêntica). Duas escolhas fora do `baseline.yaml` do PixSim: *burn-in* de 30 dias em vez de 90 (30 dias cobrem a maior janela do motor) e 1.000 eventos de fraude por mês em vez de 2.000 (calibrados para 100 mil contas). A fraude é superamostrada de propósito, e os custos "ponderados" aplicam o peso de importância.

5.4 Adaptador: o que foi mapeado, inventado e não existe

Campo do contrato	Origem
`amount`	valor em R$ do PixSim para centavos inteiros (`Decimal`, meio para cima, mínimo 1 centavo)
`transaction_date`	início fixo 2026-09-01T00:00-03:00 mais o tempo em segundos do PixSim; o *burn-in* tem tempo negativo
`source_account`, `destination_account`	**inventados**: agência, conta e ISPB determinísticos a partir do `account_id`; `opening_date` do PixSim deslocada para a mesma origem, preservando a idade da conta
`owner.document_number`	**inventado**: CPF ou CNPJ sintético com dígitos verificadores válidos, derivado do `account_id` por mapa bijetor
`source.device_id`	o dispositivo do PixSim
`source.channel`	`mobile` → `app`, `internet` → `internet_banking`; saídas da dispersão: `crypto_exit`, `cash_withdrawal`, `merchant_purchase`
`source.ip`	**inventado**: derivado do hash do dispositivo, faixa 198.18.0.0/15
`source.geolocation`	**não existe** no PixSim: o campo opcional fica ausente e as regras e características geográficas ficam inertes

Os 254.795 payloads gerados passam no JSON Schema do enunciado (0 inválidos). Rótulos e metadados do PixSim (`typology`, `role`, `burn_in`, `weight`, `report_delay_days`) viajam no arquivo de eventos mas **não** vão ao motor.

5.5 Mundos A e B; perspectiva do banco pagador

O **mundo A** (semente 20260910) serve para *escolher* parâmetros e *treinar* modelos; o **mundo B** (semente 20261017, 259.646 Pix, 986 eventos de fraude, 1.438 Pix de vítima) serve para *confirmar* e *testar*. Os dois têm a mesma escala e vêm do mesmo gerador e da mesma configuração; só muda a semente. Portanto o teste mede generalização entre *amostras* do PixSim, não entre geradores.

Perspectiva. O motor vê todos os Pix, mas a leitura é a do banco pagador. "Pagador→mula" (`victim_to_mule`) é a fraude contra o nosso cliente e é o que o motor deve barrar. A dispersão (`mule_forward`, `cash_out`) e a semente do Pix errado só chegariam ao motor se a mula fosse cliente do mesmo banco; entram numa visão separada e não somam na captura principal nem no objetivo da calibração.

5.6 Métricas

Métrica	Definição
Fricção	parcela dos legítimos com 2FA (e reprovados)
Captura	parcela dos Pix de vítima fora de `approved`; e, por evento de fraude, parcela com ao menos um Pix de vítima barrado
Custo ponderado por 1.000 Pix	seção 2.9, com `cost-model.json` e o peso de importância
P@k diário	precisão nos $k \in \{5, 10, 25, 50\}$ Pix mais suspeitos do dia (ranking: decisão das regras como escore, ou escore do LightGBM)
AUC	pontuação do modelo na fraude contra o legítimo
Recall e fricção no limiar	no limiar do próprio modelo, escolhido por F1 máximo no pedaço de validação
PSI	deriva de características entre dois conjuntos
Latência	por Pix, em memória e pela API

Para modelos, "positivo" é o Pix de vítima; "legítimo" é o tráfego legítimo. A dispersão e a semente do Pix errado ficam fora do treino e da avaliação, porque misturá-las ensinaria o modelo a pontuar o

comportamento da mula e não o da vítima. AUC, recall e P@k são brutos (sem ponderar); custos e precisão ponderada usam o peso.

5.7 Avaliação nos dois sentidos

Para separar "aprendeu o PixSim" de "aprendeu o nosso gerador", os modelos são avaliados em **ambos os sentidos**: treinados no nosso simulador e testados no PixSim B, e treinados no PixSim A e testados no PixSim B (generalização entre sementes) e no nosso simulador (semente 7). O sentido inverso (PixSim A $\rightarrow$ nosso) mostra que os dois geradores ensinam sinais diferentes: nenhum modelo é "o" modelo e o PixSim não substitui o teste no nosso simulador.

5.8 Calibração por custo (mundo A escolhe, mundo B confirma)

O grid varre **um parâmetro por vez** a partir da linha de base (os padrões anteriores, fixados em `calibration-grid-pixsim.json`), mais a combinação dos melhores valores de cada um: 21 combinações no mundo A, ~20 passadas de 1 min. O objetivo é o menor custo ponderado por 1.000 Pix na visão do pagador; em empate, menos legítimos reprovados e depois mais sequências barradas. As metas, em ordem: (1) legítimos reprovados perto de zero; (2) custo mínimo; (3) 2FA em legítimos abaixo de 5% quando o custo empata. O script nunca altera `src/constants.py`; a escolha final e o porquê são do time (ADR 0027). Parâmetros varridos: teto diário, cauda da Poisson, corte do fracionamento, z do perfil, valor mínimo de aparelho novo, valor mínimo do primeiro Pix para um destino e limite noturno (o teto diário e o corte do fracionamento pertenciam a regras depois descartadas pela ablação, seção 4.2.7).

5.9 Alinhamento com a metodologia do PixSim e comparação com o artigo

Com base em `docs/metrics_definitions.md` do PixSim (SHA 7fdfc90), os pontos de contato são:

Aspecto	PixSim	Aqui	Situação
Perspectiva e positivos	banco pagador; Pix `victim_to_mule`, dispersão à parte	os mesmos	alinhado
Peso da cadeia	peso de importância por evento de fraude	o mesmo peso em custos e cascata	alinhado
Falso bloqueio	R$ 50 + 2% do valor	R$ 50 + 2% do valor, desde esta mudança	alinhado (antes: R$ 50 fixos)
Escala	1,08 milhão de contas, 90 dias de aquecimento	5 mil contas, 30 dias	difere
Fraude	taxa real com pesos	superamostrada; os pesos corrigem custo, não P@k	difere
Ações	`pass`, `alert` (fila), `block`	`approved`, `two_factor_required`, `rejected`	difere: sem fila de analistas
Regimes do MED de 2026, marcação no DICT	comparados	só o relógio padrão	difere
Scorer	GBM ajustado no aquecimento, avaliado na janela do mesmo mundo	treinado no mundo A, avaliado no B	difere: AUC não comparável

(Fonte: `validacao-pixsim.md`, "Alinhamento com a metodologia do PixSim".)

O resumo do artigo cita um escorer sintético com AUC de 0,82. Esse número **não é comparável** aos nossos 0,99: o desenho da avaliação é outro (escala cheia, escorer ajustado no aquecimento e avaliado na janela seguinte do mesmo mundo, com as características do PixSim). A diferença **não** se explica por ruído: o ruído de score do PixSim só existe no seu experimento de sensibilidade S7; o escorer padrão não tem ruído (`score_noise_sigma` = 0). O 0,82 serve, no máximo, como ordem de grandeza. A seção 6.14 replica o S7 sobre os nossos modelos.

5.10 Vazamento de dados e vieses do gerador

Um AUC de 0,99 pede a pergunta: o modelo está vendo a resposta? Checamos o vazamento de dados (*data leakage*) em quatro frentes e separamos o que é vazamento do que é viés do gerador. Medição *ad hoc* de 18/10/2026 sobre as features exportadas dos mundos A e B (scripts de verificação não versionados; os dados de entrada saem de `make pixsim-datasets`).

Sem vazamento de rótulo. O adaptador só passa ao motor os campos do contrato do enunciado. Os campos do PixSim que entregariam a resposta (`label`, `role`, `ring_id`, `is_mule`, `is_new_counterparty`, `hop_depth`, `sender_balance_before`) ficam fora do payload. As características vêm só do histórico *anterior* ao Pix: a paridade API × backtest (seção 4.5) deu 0 divergências, o que confirma que o backtest não olha o futuro, já que a API não tem como olhar. O score de grafo nunca entra no backtest (o worker não roda lá).

Mundos A e B independentes. Só muda a semente, mas as populações não se repetem: só 3 das 5.025 contas têm a mesma data de abertura nos dois mundos, e as mulas com o mesmo `account_id` nos dois mundos são 364, contra cerca de 359 esperadas ao acaso.

Separação no tempo. Os modelos são ajustados nos primeiros 75% do mundo A, em ordem de `transaction_date`, e o limiar é escolhido nos 25% finais; o teste é no mundo B. Na retroalimentação, o desfecho do MED só chega em `transaction_date` + atraso de reporte, nunca antes.

Valores "colados" não explicam o resultado. O PixSim gera fraudes com valores logo abaixo de números redondos (R$ 999,99, R$ 1.999,99, …): são 22% das fraudes e 0% do legítimo, um atalho que o modelo poderia decorar. O LightGBM não depende dele:

Teste (LightGBM, treino A, teste B)	AUC
Linha de base do experimento (mesma receita de treino)	0,992
Sem as fraudes de valor colado	0,990
Com ruído de $±$3% no valor, no treino e no teste	0,993

O experimento retreina o LightGBM à parte, por isso a linha de base (0,992) difere do modelo instalado (0,989, seção 6.6); o que importa é a variação dentro do experimento, de no máximo 0,002.

Vieses do gerador (não são vazamento). O AUC alto vem de premissas do PixSim, não de um atalho no nosso pipeline:

- só o valor dá AUC de 0,937, e sem nenhuma característica de valor o modelo ainda chega a 0,943; as medianas de valor da fraude no PixSim são premissas marcadas como ASSUMED (A2.2) no próprio PixSim, não medidas;
- o PixSim limita o Pix de conta tomada à noite a R$ 1.000 (assume que o banco aplica o limite noturno), então `LIMIT_NIGHT_AMOUNT` não consegue disparar nessa tipologia;
- 19% dos Pix de vítima do mundo B estão em exatamente R$ 999,99 ou R$ 1.000,00, invisíveis para `COUNTERPARTY_FIRST_TIME_HIGH`, que pede valor acima de R$ 1.000;
- o mundo B, usado na ablação e na escolha dos modelos, deixou de ser teste puro; por isso o descarte das três regras (seção 4.2.7) foi confirmado também no mundo A.

5.11 O que o PixSim não gera

O PixSim não gera geolocalização, IP, fazenda de aparelhos (um aparelho por conta), rajada na conta da vítima, funil de muitas vítimas na mesma hora para a mesma mula, nem fracionamento; e o worker não roda no backtest. As regras desses sinais (`PROFILE_GEO_VELOCITY`, `VELOCITY_SOURCE_COUNT`, `VELOCITY_DEVICE_SOURCES`, `COUNTERPARTY_FAN_IN`, `COUNTERPARTY_GRAPH_RISK` e a parte geográfica de `TYPOLOGY_ACCOUNT_TAKEOVER`) nunca disparam lá: ficam pelo critério 3 da ADR 0033 (cobrem um sinal que o PixSim não gera, com custo zero nele). A evidência delas é outra: testes pela API (um que dispara e um que não dispara, por regra) e a trava do CI no nosso simulador, que gera esses padrões. Não

inventamos geolocalização no adaptador, porque mediríamos a nossa própria invenção, o mesmo erro circular da seção 5.1.

6 Resultados no PixSim

Todas as medidas desta seção são **métricas no PixSim**, em escala reduzida, com uma semente por mundo e sem intervalo de confiança. Salvo menção, valem para o conjunto final (`2026.10.18-pixsim-2`, 15 regras, parâmetros calibrados, falso bloqueio a R$ 50 + 2%) no mundo A. Diferenças de custo abaixo de cerca de R$ 1 por 1.000 Pix são ruído nesta escala.

6.1 Fricção em legítimos

Conjunto	Legítimos	`approved`	2FA	`rejected`
PixSim A	125.518	97,03%	2,97%	0,00%
PixSim B	127.683	96,91%	3,09%	0,002%

(Fonte: `validacao-pixsim.md`, "Fricção em legítimos" e "Contribuição de cada regra".) A meta de 2FA abaixo de 5% foi cumprida nos dois mundos, e nenhum legítimo do mundo A é reprovado.

6.2 Captura por tipologia

Tipologia	Pix de vítima	Aprovado (escapou)	2FA	Reprovado	Eventos com ao menos um Pix barrado
Conta tomada	323	22,60%	73,07%	4,33%	206/209 (98,6%)
Engenharia social	898	53,12%	46,88%	0,00%	350/618 (56,6%)
QR adulterado	68	72,06%	27,94%	0,00%	19/68 (27,9%)
Pix errado	91	90,11%	9,89%	0,00%	9/91 (9,9%)
Total (vítima)	1.380	49,3%			584/986 (59,2%)

(Fonte: `validacao-pixsim.md`, "Captura por tipologia" e "Sequências e anéis"; a linha de total é derivada: 50,7% dos Pix de vítima barrados.) Conta tomada é a tipologia mais barrada; engenharia social, Pix errado e QR adulterado escapam em boa parte porque o Pix da vítima parece um pagamento comum, dentro do perfil dela. Em valor, só 1,7% do dinheiro das vítimas é reprovado, 70,2% cai em 2FA (que a vítima coagida pode confirmar) e 28,2% passa aprovado. Na visão da dispersão (mula movendo o dinheiro), 70,9% dos eventos têm ao menos um Pix barrado em qualquer papel (699/986), e todos os 75 anéis de mulas têm ao menos um Pix barrado (75/75); a parte da dispersão só ajudaria se a mula fosse cliente do banco.

Regras que disparam em legítimos são raras: a mais frequente é `COUNTERPARTY_FIRST_TIME_HIGH` (1,2%), seguida de `COUNTERPARTY_NEW_ACCOUNT` (1,2%) e `LIMIT_NIGHT_AMOUNT` (0,6%); em Pix de vítima, `COUNTERPARTY_FIRST_TIME_HIGH` dispara em 33,0%, `PROFILE_NEW_DEVICE` em 14,9% e `VELOCITY_POISSON_GAMMA` em 6,4%. `TYPOLOGY_MULE`, `COUNTERPARTY_FAN_IN` e `COUNTERPARTY_NEW_ACCOUNT` ficam em zero em Pix de vítima: no PixSim as mulas são contas com idade comum, e as vítimas chegam à mesma mula espalhadas no tempo, não em rajada (hipótese; não medimos a distribuição temporal). *(Fonte: `validacao-pixsim.md`, "Regras que disparam" e "O que se mantém e o que cai".)*

6.3 Custo esperado

Visão	Custo ponderado por 1.000 Pix	Parte fraude	Parte fricção
Visão pagador (legítimo + pagador→mula)	R$ 109,61	R$ 50,12	R$ 59,49
Visão dispersão (legítimo + dispersão)	R$ 212,59	R$ 153,11	R$ 59,48

(Fonte: `validacao-pixsim.md`, "Custo esperado (V1").) O custo bruto (sem ponderar) da visão pagador é R$ 7.023,16 por 1.000 Pix, inflado pela superamostragem. A fricção responde por cerca de 54% do custo ponderado na visão pagador: com a taxa de fraude real, o atrito sobre o legítimo pesa tanto quanto a fraude que passa.

6.4 P@k diário

Ranking	P@5	P@10	P@25	P@50
Visão pagador, regras	13,3%	16,7%	21,7%	22,9%
Visão pagador, regras + LightGBM (dentro da amostra)	100,0%	98,7%	89,1%	70,1%
Visão dispersão, regras	11,0%	17,7%	28,0%	29,7%
Visão dispersão, regras + LightGBM (dentro da amostra)	71,0%	57,1%	43,9%	35,8%

(Fonte: `validacao-pixsim.md`, "P@k diário (V1").) **Cuidados.** A fraude está superamostrada e a precisão fica inflada na proporção do peso; com a taxa real, a precisão ponderada dos alertas fica abaixo de 1% e a fila dos analistas seria quase toda de falso positivo. Além disso, a linha "regras + LightGBM" usa o LightGBM instalado, treinado no mundo A, **no próprio mundo A** (dentro da amostra); fora da amostra, no mundo B, o mesmo ranking dá P@5 de 96,7% e P@50 de 57,9% (seção 6.6).

6.5 Calibração: efeito de cada parâmetro e configuração final

A calibração foi feita **antes** do descarte das três regras: grid no mundo A com as 18 regras de então e falso bloqueio a R$ 50 fixos (*fonte:* `calibracao.md`). Efeito de cada parâmetro que continua no motor, sozinho, sobre a linha de base (custo R$ 143,58 por 1.000 Pix; 3,50% de 2FA em legítimos; 36,1% dos Pix de vítima barrados; 383 de 986 eventos barrados):

Mudança	Custo R$/1.000	2FA legítimos	Pix de vítima barrados	Eventos barrados
Primeiro Pix para destino: R$ 3.000 → **R$ 1.000**	**135,72**	4,16%	52,2%	577
Limite noturno R$ 1.000 → R$ 2.000 (não aplicado)	139,01	3,18%	34,4%	365
Cauda Poisson 100 → **30** ppm	142,30	3,30%	33,3%	368
z do perfil 3,5 → **4,5**	142,33	3,35%	35,2%	376
Aparelho novo R$ 500 → **R$ 200**	142,91	3,50%	38,8%	413
Cauda Poisson 100 → 1.000 ppm	166,48	5,03%	42,0%	426
Limite noturno R$ 1.000 → R$ 500	152,77	4,10%	40,9%	434

O valor mínimo do primeiro Pix para um destino é o parâmetro de maior efeito no custo. O grid também varria o teto diário e o corte do fracionamento, que saíram do código com as suas regras (seção 4.2.7); as linhas deles ficam registradas em `calibracao.md`.

Configuração final. Ficam no motor quatro parâmetros calibrados: primeiro Pix para destino novo a partir de R$ 1.000, aparelho novo a partir de R$ 200, cauda da Poisson em 30 ppm e z do perfil acima de 4,5. O efeito de cada etapa, com o mesmo modelo de custo (R$ 50 + 2%):

Mundo	Etapa	Custo R$/1.000 Pix	Pix de vítima barrados	2FA em legítimos
B	15 regras, parâmetros anteriores à calibração	127,26	37,4%	
B	18 regras, parâmetros calibrados	153,51	51,8%	4,13%

Mundo	Etapa	Custo R$/1.000 Pix	Pix de vítima barrados	2FA em legítimos
B	**15 regras, parâmetros calibrados (final)**	**118,82**	51,1%	3,09%
A	18 regras, parâmetros calibrados	132,13	51,7%	3,81%
A	**15 regras, parâmetros calibrados (final)**	**109,61**	50,7%	2,97%

(Fonte: `validacao-pixsim.md`, "Contribuição de cada regra"; ADR 0033.) No conjunto final, a calibração tira R$ 8,44 por 1.000 Pix no mundo B e leva os Pix de vítima barrados de 37,4% para 51,1%. Na escolha original (18 regras, R$ 50 fixos) o ganho foi de R$ 11,92 no mundo A e R$ 8,36 no B; o ganho menor em B é a melhor estimativa de quanto a escolha é específica de um mundo.

Limite noturno mantido por âncora regulatória. O grid preferia R$ 2.000 para `LIMIT_NIGHT_AMOUNT`, mas o parâmetro ficou em **R$ 1.000**, o limite noturno padrão de pessoa física do BCB (Res. 142/2021) [11]: subir o corte pararia de pedir 2FA abaixo do teto que o regulador associa ao risco de sequestro relâmpago. Com o conjunto final, R$ 2.000 custaria R$ 114,02 no mundo B contra R$ 118,82 (+R$ 4,80 por 1.000 Pix pela âncora). O parâmetro é `pinned` no grid: medido, nunca escolhido.

Sensibilidade (mundo A). A análise 4 de `analises.md` varre a cauda da Poisson e o z do perfil com os demais parâmetros nos padrões atuais. `RULE_POISSON_TAIL_PPM` tem o maior efeito (R$ 24,51 de amplitude de custo; 1,81 p.p. de fricção; 9,57 p.p. de fraude barrada) e `RULE_PROFILE_Z_2FA_TENTHS` o menor (R$ 11,27; 1,00 e 4,13 p.p.). Onde a fraude barrada não muda com a fricção, o limiar está num platô e a escolha é por custo de falso positivo; onde as duas sobem juntas, há troca real. A sensibilidade no mundo B está na seção 6.12.

6.6 Modelos em *shadow*

Treino no mundo A, teste no mundo B e no nosso simulador (semente 7), com as características do conjunto `2026.10.18-pixsim-2`. Hiperparâmetros fixos de `train_models.py`; ajuste nos primeiros 75% do mundo A no tempo (95.173 linhas), validação nos 25% finais (31.725; limiar por F1 máximo; Platt na validação). Limiares: logreg 0,1933, LightGBM 0,8558, Isolation Forest 0,6035.

AUC e decisão no mundo B:

Decisor	AUC nosso→PixSim B	AUC PixSim A→B	Recall A→B	Fricção A→B	Custo A→B (R$/1.000)
Regressão logística	0,770	0,990	64,7%	0,8%	53,39
LightGBM	0,580	0,989	73,0%	1,0%	59,12
Isolation Forest	0,788	0,850	8,0%	1,4%	157,88
Regras	0,740	0,740	51,1%	3,1%	118,82
Regras + LightGBM	0,718	0,987	80,9%	3,6%	99,42

(Fonte: `validacao-pixsim.md`, "Modelos do caminho quente".) "Regras + LightGBM" é a política em que as regras decidem e um score do LightGBM acima do limiar sobre um Pix aprovado vira 2FA: barra 80,9% dos Pix de vítima e custa R$ 99,42, contra R$ 118,82 das regras sozinhas ($-$16%). Os modelos sozinhos custam menos ainda, mas sem o motivo explicável que o enunciado pede. No sentido inverso (PixSim A → nosso simulador) o LightGBM cai a AUC 0,775 e 22,2% de recall, e as regras (AUC 0,965) ganham dos modelos. Captura por tipologia no mundo B com os modelos treinados no PixSim A: conta tomada 84,6% (LightGBM) e 91,9% (logreg); engenharia social 74,1% e 60,3%; QR adulterado 38,8% e 28,7%; Pix errado 27,8% e 16,7%.

GraphSAGE (nível de nó, grafo pagador→destino em janelas de 72 h; rótulo de nó = `is_mule` do PixSim, com 1.366 mulas no A e 1.321 no B):

Modelo	Avaliado em	Nós alvo	AUC	Precisão	Recall	Fricção em destinos legítimos
Treinado no PixSim A	treino	80.709	0,912	0,777	0,754	2,8%
Treinado no PixSim A	PixSim B	82.682	0,919	0,786	0,787	2,5%
Treinado no nosso simulador	PixSim B	82.682	0,518	0,241	0,005	0,2%

Leituras. (i) Trocar a fonte de treino leva logreg e LightGBM de AUC 0,77 e 0,58 para 0,99. (ii) O 0,99 é otimista: os mundos A e B vêm do mesmo gerador, então mede generalização entre sementes, não independência; a seção 5.10 mostra que não há vazamento de rótulo, mas que parte do sinal vem de premissas do gerador. (iii) Pix errado (17% a 28% de captura) e QR adulterado (29% a 39%) continuam difíceis. (iv) O Isolation Forest treinado no PixSim melhorou o AUC (0,85 contra 0,79), mas o limiar p99 só captura 8,0%; o critério de troca não foi atendido e o `iforest.json` permanece o do simulador. (v) O GraphSAGE de AUC 0,92 é um teto: o rótulo é estático por conta e a janela contém os Pix pontuados. (vi) O HMM não foi avaliado no PixSim. Os JSON de logreg, LightGBM e GraphSAGE instalados no repositório vêm do PixSim A e seguem em *shadow*; o Isolation Forest e o HMM continuam os do nosso simulador; o `graphsage.json` do PixSim pontua abaixo de uma conta comum a mula do nosso simulador (conta nova, 10 pagadores), porque a mula do PixSim é uma conta antiga que recebe de vários pagadores e dispersa, e o teste de integração do worker foi reescrito para montar esse cenário (a asserção não mudou).

Sobre a comparação com o artigo: ver a seção 5.9; o AUC de 0,82 do escorer sintético do artigo não é comparável ao nosso 0,99.

6.7 Rótulos atrasados

O corte de treino T é o fim do mundo A; uma fraude só vira positiva se `transaction_date + atraso de reporte` $\leq T$ e se foi reportada; as demais entram como legítimas. LightGBM com os mesmos hiperparâmetros, avaliado no mundo B com rótulos verdadeiros:

Cenário de rótulo	Positivos conhecidos	AUC	Recall	Fricção	Custo R$/1.000
Oráculo (todos os rótulos)	1.380 de 1.380	0,989	73,0%	1,0%	59,12
Atraso do PixSim (MED)	1.083 de 1.380	0,968	65,4%	0,7%	71,06
Atraso fixo de 3 dias	1.254 de 1.380	0,989	61,6%	0,5%	62,68
Atraso fixo de 7 dias	1.061 de 1.380	0,989	63,0%	0,5%	61,78
Atraso fixo de 14 dias	723 de 1.380	0,657	11,3%	0,9%	159,56
Atraso do PixSim, descartando os últimos 14 dias	644 de 1.380	0,957	49,4%	0,5%	86,10

(Fonte: `validacao-pixsim.md`, "Rótulos atrasados".) Com o atraso real do MED (mediana de 2 dias, cauda de 17) o modelo perde pouco no AUC (0,989 para 0,968) e algum recall (73,0% para 65,4%), mas o custo sobe cerca de 20% (R$ 59,12 para R$ 71,06). O dano cresce rápido com o atraso fixo: 14 dias deixam o

modelo quase cego (AUC 0,657, recall 11,3%), porque os positivos recentes, mais parecidos com o tráfego de amanhã, viram "legítimos". Descartar os últimos 14 dias não recuperou nada com um mundo de 30 dias (R$ 86,10: jogou fora metade dos positivos). Na prática, retreinar com frequência e esperar a cauda do MED importa mais do que corrigir o prior.

6.8 Retroalimentação pelo MED

Simulada **só no backtest** (a API, o banco e as regras ativas não mudam; ADR 0032). O mundo B é reproduzido em ordem de `transaction_date`; cada fraude de vítima reportada gera um desfecho em `transaction_date` + atraso de reporte, aplicado ao histórico quando o relógio dos dados o alcança. Pix reprovado não liquida e não gera desfecho (viés de seleção); com 2FA, 10% escapam e geram desfecho. Peças: regra `COUNTERPARTY_CONFIRMED_FRAUD` (destino que recebeu fraude confirmada de outro pagador antes de t, pedindo 2FA, escolhido no mundo A por custar R$ 103,47 por 1.000 Pix contra R$ 225,28 de reprovar), LightGBM instalado como regra extra e retreino a cada 7 dias com promoção *champion/challenger*.

Cenário	Custo R$/1.000	Captura	Fricção em legítimos	Fraudes sem desfecho possível	Escapes confirmados
a) regras, sem retroalimentação	118,82	51,1%	3,1%	47,6%	597
b) + sinal de fraude confirmada	112,23	65,4%	3,3%	59,9%	438
c) + modelo campeão fixo	100,84	84,5%	3,8%	77,4%	195
d) + retreino semanal	103,18	80,7%	3,7%	73,9%	233

(Fonte: `validacao-pixsim.md`, "Retroalimentação pelo MED".) Leituras: (1) o sinal de fraude confirmada é o ganho mais barato: −R$ 6,59 por 1.000 Pix e +14 pontos de captura; (2) o retreino semanal **piorou** o resultado: dois desafiantes custaram menos na semana de sombra e foram promovidos (semanas 1 e 2), mas nas semanas seguintes custaram mais do que o campeão original custaria, e o cenário fecha em R$ 103,18 contra R$ 100,84 do campeão fixo. Uma semana de sombra com fraude superamostrada é pouco para separar desafiante melhor de sorte, o mundo é estacionário e os rótulos maduros vêm censurados pelo próprio motor; (3) **viés de seleção**: 47,6% das fraudes (ponderadas) já não geram desfecho com as regras, e 77,4% com o campeão decidindo; a precisão do 2FA vista pelos desfechos (0,01% a 0,04% por semana) fica cerca de dez vezes abaixo da real (0,15% a 0,23%), e a de reprovação é zero por construção, então monitorar só por rótulo subestima o que o motor acerta; (4) limites: 4 retreinos em 30 dias, fraude superamostrada e custo semanal ruidoso.

6.9 Peer group: medido e descartado

`PROFILE_PEER_AMOUNT` comparava o valor de um pagador sem perfil próprio (menos de 5 Pix em 30 dias) com os de pares do mesmo grupo (PF/PJ × idade da conta; até 500 valores, mínimo 50), com z robusto $> 3{,}5$ (nível 2 de Bolton & Hand). Medida com as 18 regras e os parâmetros anteriores à calibração: antes e depois, no mundo B e no nosso simulador, fricção, captura e custo foram **idênticos** (R$ 157,60 por 1.000 Pix no B): só 12 dos 1.438 Pix de vítima (0,8%) vêm de pagador sem perfil, com z mediano de 1,7, abaixo do corte. A latência pela API (300 mil Pix em 30 dias) seria +9 ms com grupo cheio e +94 ms com grupo vazio. A regra **não entra no motor** (ADR 0028); o pagador novo continua coberto pelas regras de payload e de velocidade fixa.

6.10 Latência

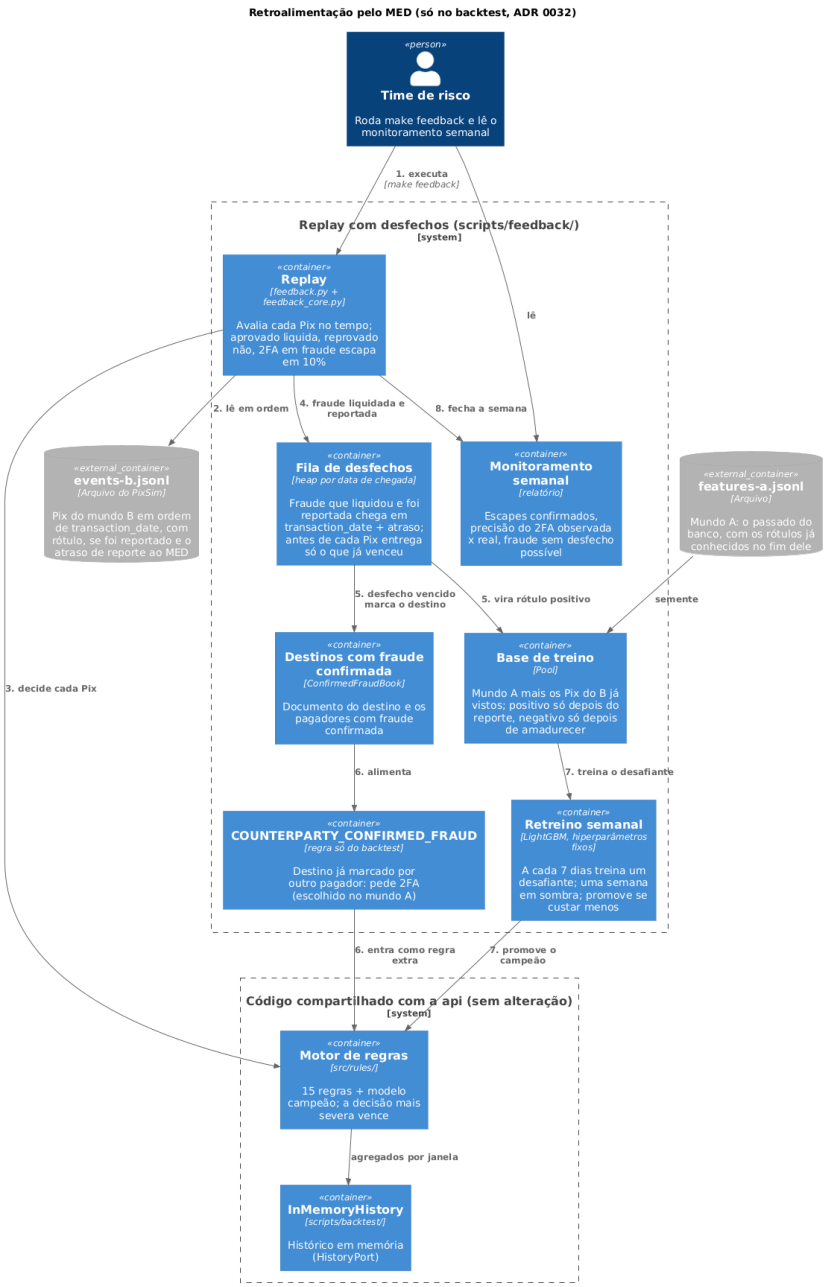


Figura 6: Retroalimentação pelo MED no backtest: containers e passos numerados do ciclo.

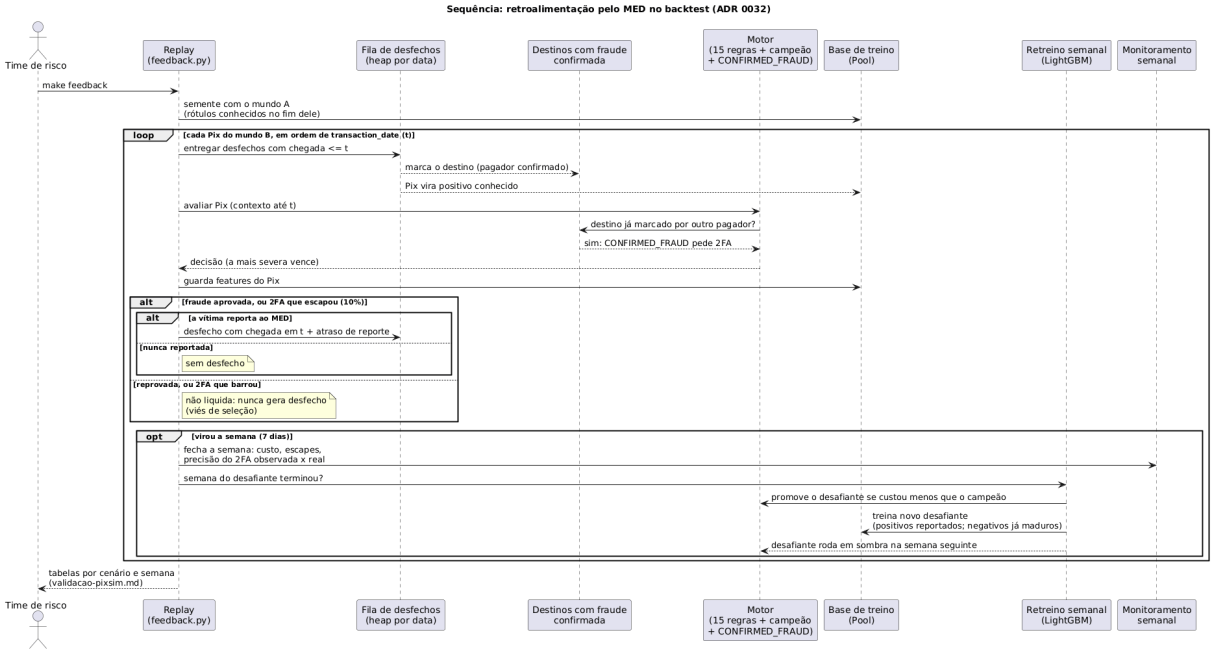


Figura 7: Sequência da retroalimentação: os desfechos vencidos são entregues antes de avaliar cada Pix, e Pix reprovado nunca gera desfecho.

Medida	Valor	Fonte
Motor em memória (sem HTTP), 254.795 Pix	P50 0,33 ms; P95 0,60 ms; P99 0,84 ms; 2.710 Pix/s	`validacao-pixsim.md`, "Latência do backtest"
Backtest em lote completo	101,7 s (2.506 Pix/s)	idem
Três modelos em *shadow*	+149 μs por Pix	ADR 0024
Pela API, 300 mil Pix em 30 dias	média de 50 ms por Pix com histórico e 59 ms por pagador novo (medida no experimento do grupo de pares)	`validacao-pixsim.md`, "Peer group"

A latência do motor é dominada pelas consultas de histórico, não pelas regras; os agregados são *range scans* em índice (seção 3.5).

6.11 Deriva das características

O PSI do nosso gerador contra o PixSim é muito alto em características geográficas (8,3, porque o PixSim não tem geolocalização) e de histórico (o *burn-in* de 30 dias contra 14 dias do nosso), o que é artefato de construção. O que sobra de informação real: valor (`amount`, PSI 0,74), idade da conta de origem (0,87) e, em Pix de vítima, `destination_age_days` (4,39, coerente com `COUNTERPARTY_NEW_ACCOUNT` em zero no PixSim). Entre os mundos A e B, 0 características passam de 0,1 considerando todos os Pix e considerando só a fraude: duas amostras do mesmo gerador não derivam (*fonte: `validacao-pixsim.md`, "PSI"; `analises.md`, análise 3*).

6.12 Contribuição de cada regra (ablação)

Mundo B, visão pagador, 127.683 legítimos e 1.438 Pix de vítima, custo com falso bloqueio de R$ 50 + 2%, versão `2026.10.18-pixsim-2`. Linha de base com as 15 regras: **R$ 118,82 por 1.000 Pix**, 51,1% dos Pix de vítima barrados, 3,09% de 2FA e 0,00% de reprovação em legítimos. "Contribuição" é o custo sem a regra (as outras 14 ligadas, `backtest.py --disable`) menos o custo com todas: positiva, a regra economiza. A ablação é marginal: duas regras que cobrem o mesmo Pix aparecem ambas com contribuição baixa.

Regra	Disparo em legítimos	Disparo em vítimas	Contribuição (R$/1.000 Pix)	Vítima barrada sem a regra	Veredito
COUNTERPARTY_FIRST_TIME_HIGH	1,28%	32,7%	+34,63	27,7%	contribui
PROFILE_NEW_DEVICE	0,002%	15,0%	+5,60	40,8%	contribui
VELOCITY_POISSON_GAMMA	0,14%	6,8%	+3,77	45,3%	contribui
TYPOLOGY_ACCOUNT_TAKEOVER	0,000%	1,0%	+0,31	51,1%	contribui
PROFILE_NEW_SOURCE_ACCOUNT	0,006%	0,1%	+0,18	51,0%	contribui
VELOCITY_SOURCE_COUNT, VELOCITY_DEVICE_SOURCES, PROFILE_GEO_VELOCITY, COUNTERPARTY_FAN_IN, COUNTERPARTY_GRAPH_RISK	0,00%	0,0%	0,00	51,1%	nunca dispara no PixSim
LIMIT_SINGLE_AMOUNT	0,022%	0,0%	$-$0,22	51,1%	ruído (único teto de exposição por Pix)
TYPOLOGY_MULE	0,001%	0,0%	$-$0,39	51,1%	custa mais com ela (âncora: art. 39-B, §1º, II)
PROFILE_AMOUNT_ROBUST_Z	0,112%	1,0%	$-$1,54	50,9%	custa mais com ela (âncora: Iglewicz & Hoaglin)
LIMIT_NIGHT_AMOUNT	0,568%	4,6%	$-$6,52	50,2%	custa mais com ela (âncora: Res. BCB 142/2021)
COUNTERPARTY_NEW_ACCOUNT	1,00%	1,1%	$-$25,38	50,6%	custa mais com ela (âncora: art. 39-B, §1º, II)

(Fonte: `validacao-pixsim.md`, "Contribuição de cada regra", regerada com as 15 regras.) **Leitura.** Cinco regras pagam o próprio custo no PixSim, e uma delas (`COUNTERPARTY_FIRST_TIME_HIGH`) responde por quase tudo: sem ela, os Pix de vítima barrados caem de 51,1% para 27,7%. Cinco nunca disparam, porque o PixSim não gera o sinal que elas procuram (seção 5.11); `TYPOLOGY_MULE` dispara uma única vez, num legítimo, e `LIMIT_SINGLE_AMOUNT` em 28 legítimos. Das cinco de contribuição negativa, quatro ficam por âncora regulatória ou estatística (critério 2 da ADR 0033). `LIMIT_SINGLE_AMOUNT` fica por outro motivo (ADR 0033): com 18 regras a contribuição era nula, os R$ 0,22 de agora estão abaixo de R$ 1 por 1.000 Pix, que nesta escala é ruído, e ela é o único teto de exposição por Pix do motor. A maior negativa, `COUNTERPARTY_NEW_ACCOUNT`, custa mais porque as mulas do PixSim são contas de idade comum e a regra pede 2FA a legítimos que pagam contas novas. O PixSim mede o que o seu gerador desenhou: ausência de efeito aqui não prova ausência de efeito em Pix real. As três regras que tinham contribuição negativa sem âncora saíram do motor (seção 4.2.7).

Sensibilidade (mundo B, um parâmetro por vez). O parâmetro de maior efeito é a idade mínima da conta de destino (`RULE_DESTINATION_MIN_AGE_DAYS`, compartilhado com `TYPOLOGY_MULE`): 15 dias custariam R$ 102,73 e 60 dias R$ 164,54, contra R$ 118,82 com 30. O valor mínimo do primeiro Pix para um destino troca captura por fricção: R$ 500 barra 71,7% dos Pix de vítima a R$ 125,85; R$ 3.000 barra 36,1% a R$ 124,23. O limite noturno de R$ 2.000 custaria R$ 114,02, mas o corte fica no limite regulatório de R$ 1.000. Baixar o mínimo de pagadores de `TYPOLOGY_MULE` de 5 para 3 custaria R$ 131,35. Os demais parâmetros mexem o custo em no máximo R$ 2,00. Nenhum parâmetro mudou por esta medida, que é do mundo B, e não do mundo que escolhe.

6.13 Cascata de valor

Roda o relógio do MED do próprio PixSim sobre as decisões do nosso motor no mundo B regerado. `approved` liquida; `rejected` bloqueia; `two_factor_required` em fraude bloqueia com probabilidade 0,9 e liquida no restante; em legítimo, o 2FA é confirmado e liquida. "Perdido" é a perda de valor de vítima depois da recuperação pelo MED e pelo bloqueio cautelar.

Política	Barrado antes	Retido num salto	Recuperado pelo MED	Recuperado por bloqueio cautelar	Perdido	Falsos bloqueios	Dano dos falsos bloqueios
`pass_all` (nosso mundo B)	0,0%	0,0%	6,1%	14,6%	79,3%	0	R$ 0
Regras	66,1%	6,6%	1,9%	4,7%	20,7%	2	R$ 513
Regras + LightGBM decidindo	86,6%	1,5%	0,4%	2,5%	9,0%	2	R$ 513
`threshold_block` do PixSim (escorer deles)	68,5%	26,1%	0,4%	7,0%	3,0%	485	R$ 33.487

(Fonte: `validacao-pixsim.md`, "Cascata de valor".) Sem barrar nada, 79% do valor se perde; as regras sozinhas deixam perder 21%, e com o LightGBM decidindo, 9%. O `threshold_block` do PixSim perde só 3%, mas com 485 falsos bloqueios contra 2 das regras (eram 36 antes do descarte das três regras, quase todos do `TYPOLOGY_SPLITTING`). Esta linha vem de outro mundo gerado, com o escorer deles (AUC de vítima 0,998), e serve de ordem de grandeza, não de comparação Pix a Pix. O custo em reais da seção 2.9 trata fraude aprovada como perda total do valor; a cascata mostra que parte volta, então o custo é um limite superior da perda.

6.14 Sensibilidade à qualidade do escorer

Réplica do experimento S7 do PixSim sobre os modelos que já temos, sem retreinar: o score ganha ruído gaussiano no logit, $\text{logit}(p') = \text{logit}(p) + \mathcal{N}(0, \sigma)$. O ruído só existe neste experimento, não nos nossos modelos nem no escorer padrão do PixSim. Mundo B, as regras decidem e o modelo acima do limiar sobre um Pix aprovado vira 2FA; as regras sozinhas custam R$ 118,82.

σ (LightGBM)	AUC ponderada	Custo por 1.000 Pix	Fricção em legítimos	Custo com limiar de mesma fricção
0	0,989	R$ 99,42	3,6%	R$ 99,42
3	0,978	R$ 111,68	4,0%	R$ 108,85
6	0,924	R$ 167,07	6,7%	R$ 120,00
8	0,873	R$ 246,64	10,6%	R$ 123,73
12	0,788	R$ 414,65	19,0%	R$ 127,81

(Fonte: `validacao-pixsim.md`, "Sensibilidade à qualidade do scorer".) O AUC de 0,82 corresponde a $\sigma \approx 10{,}3$ para o LightGBM (e $\approx 7{,}2$ para a logística). Nesse ponto, com o limiar reajustado, o custo é R$ 127,08 (LightGBM) e R$ 122,93 (logística): **sem ganho sobre as regras** (R$ 118,82); com o limiar original do modelo, a fricção em legítimos vai a 15,7% e 16,8%. O ganho do modelo (cerca de $-$16% sem ruído) só existe enquanto o AUC fica acima de cerca de 0,93; um escorer de 0,82 não paga a fricção que causa. Ressalva: o ruído é gaussiano e independente do Pix, o que degrada o score de forma mais uniforme do que um modelo treinado com características piores.

7 Limitações e trabalhos futuros

Limitações medidas ou conhecidas.

- **Dado sintético.** O PixSim reflete hipóteses dos autores, calibradas com dados agregados do BCB, não casos reais. Os números medem fricção e custo *relativo* entre configurações.
- **Uma geração, duas sementes.** Os mundos A e B vêm do mesmo gerador; a generalização medida é entre amostras. Não há intervalo de confiança; só uma semente por mundo.
- **Taxa de escape do 2FA.** Premissa de 10% sem fonte; 69,6% do valor das vítimas cai em 2FA, e acima de ~40% de escape o motor custa mais do que aprovar tudo (`validacao-pixsim.md`, "Sensibilidade à taxa de escape do 2FA").
- **Escala reduzida.** 5.025 contas e 30 dias, contra 1,08 milhão de contas no artigo; fraude superamostrada (o P@k é inflado).
- **Sinais que o PixSim não gera.** Geolocalização, IP, fazenda de aparelhos, rajada na conta da vítima, funil de vítimas na mesma hora e fracionamento não existem no PixSim (seção 5.11); `PROFILE_GEO_VELOCITY`, `VELOCITY_SOURCE_COUNT`, `VELOCITY_DEVICE_SOURCES`, `COUNTERPARTY_FAN_IN`, `COUNTERPARTY_GRAPH_RISK` e a parte geográfica da tipologia de conta tomada só têm como evidência os testes pela API e o nosso simulador. Documento, IP, agência e conta são sintéticos.
- **Vieses do gerador.** Sem vazamento de rótulo detectado (seção 5.10), mas o AUC de 0,99 vem de premissas do PixSim (medianas de valor da fraude assumidas; só o valor já dá 0,937); o limite noturno embutido na conta tomada e os 19% de Pix de vítima em exatamente R$ 999,99 ou R$ 1.000,00 deixam duas regras cegas por construção; e o mundo B, usado na ablação e na escolha dos modelos, não é mais teste puro.
- **Família Montante sem regra.** A soma diária foi descartada (seção 4.2.7); fracionamento real só é coberto em parte pelo teto por Pix, pelo z robusto e, no nosso simulador, pelo 2FA de outras regras.
- **Só o lado do banco pagador.** A dispersão foi tratada à parte, pois exige a mula ser cliente.
- **Cold path fora do backtest.** Grafo, GraphSAGE e HMM não rodaram no backtest em lote do PixSim (os scores são `None`); o HMM mostrou pouco sinal sem eventos de sessão.
- **Parâmetros de custo são hipóteses de negócio** (ADR 0030), e o custo absoluto depende deles.
- **Paridade em amostra.** API e backtest comparados em 500 Pix.

Trabalhos futuros.

1. **Rota de desfechos do MED.** `POST /pix/transaction/{id}/outcome` e uma tabela `pix_outcome` fecham o ciclo de aprendizado em produção (ADR 0032); saem do contrato do enunciado e por isso foram simuladas só no backtest. O sinal de fraude confirmada é o primeiro candidato a ir para a API.
2. **Feature store em escala.** O histórico por SQL com índices escala até algumas centenas de milhares de Pix; em dezenas de milhares de Pix por segundo a pesquisa aponta *streaming* (o SCARFF [10] usa Kafka, Spark Streaming e Cassandra; a ADR 0025 cita Kafka, Flink e Redis como alternativa) para manter janelas deslizantes.
3. **Escala do banco.** Particionamento por data e consultas de grupo vazio (+94 ms) pedem tratamento antes de volumes reais.
4. **Rótulos reais.** Substituir o PixSim por desfechos do MED e contestações transforma o mesmo processo de backtest em backtest real e permite calibrar o prior, o limiar e o custo de verdade.
5. **Eventos de sessão.** Login, troca de limite e cadastro de chave dariam sentido ao HMM ou a uma rede recorrente.
6. **Zona cinzenta por modelo de linguagem.** A pesquisa descreve um avaliador semântico de passagem única para os casos atípicos que passam pelas regras e pelo grafo; fora do escopo e da latência desta entrega (ADR 0017).
7. **Inferência de rejeitados.** Corrigir o treino pelo viés de seleção das fraudes barradas.

8 Como reproduzir

Pré-requisitos: Docker com `docker compose`, Git, Python 3.11 (ou `uv`) e, para este PDF, `pandoc` (3.x) e `xelatex` com as fontes FreeSerif e DejaVu.

```
git clone <repositorio> && cd Hackathon
make all                  # sobe api, db e worker, espera healthy, cria .venv 3.11 e
 ↪  roda o pytest
make reset check          # recria o banco, lint (flake8) e testes; precisa passar
 ↪  antes de qualquer entrega
```

A API responde em `http://localhost:3000`; toda rota, menos `/` e `/health_check`, exige `INTERNAL-TOKEN: default_token`.

```
curl -X POST http://localhost:3000/pix/transaction \
  -H "INTERNAL-TOKEN: default_token" -H "Content-Type: application/json" \
  -d @docs/03-api/exemplo-pix.json
curl http://localhost:3000/pix/transaction/pix-000123 -H "INTERNAL-TOKEN:
 ↪  default_token"
```

`make` sem argumento lista todos os alvos. Os do dia a dia:

Comando	Faz
`make up`, `make down`, `make logs`	sobe api, db e worker (religa o worker parado pelo `make test`); para; acompanha o log
`make test`, `make lint`, `make check`	os 113 testes *black box* (com o worker parado; os testes chamam um ciclo); `flake8`; os dois
`make reset`, `make rebuild`	recria o banco (depois de mudar `database/database.sql`); reconstrói a imagem (depois de mudar `requirements.txt`)
`make pdf`	compila este relatório

Ferramentas de risco (rodam fora da API; as do PixSim precisam de `make pixsim-datasets` antes):

Comando	Faz
`make pixsim`	clona o PixSim (SHA fixo), cria o venv próprio e gera um mundo reduzido (A por padrão; B com `PIXSIM_SEED=20261017 PIXSIM_OUT=scripts/out/pixsim/data-b`)
`make pixsim-datasets`	gera os mundos A e B (se faltarem) e exporta as características de treino e teste
`make validate-pixsim`	adaptador, backtest e relatório (atualiza `docs/05-motor/validacao-pixsim.md`)
`make rule-ablation`	ablação por regra e sensibilidade no mundo B (retoma do cache; atualiza `validacao-pixsim.md`)
`make calibrate-pixsim`	grid pelo custo ponderado: escolhe no A, confirma no B
`make pixsim-parity`	paridade API × backtest em 500 Pix do PixSim (zera o banco)
`make analyze`	análises estatísticas (`docs/05-motor/analises.md`)
`make train-models`, `make train-graphsage`, `make train-hmm`	retreina os modelos em *shadow* e instala os JSON em `src/rules/models/artifacts/` e `src/worker/models/artifacts/`
`make model-parity`	compara os avaliadores em Python puro com `scikit-learn` e `lightgbm`
`make delayed-labels`, `make feedback`	rótulos atrasados e retroalimentação pelo MED (atualizam `validacao-pixsim.md`)
`make simulate`, `make parity`, `make calibrate-check`	nosso simulador, a paridade dele e a trava de regressão do CI (fixture, sem valor estatístico); `make reset` depois

Não rode o simulador enquanto a suíte de testes roda: os testes zeram o banco. `make test` para o worker para que os testes controlem cada ciclo; `make up` religa.

Do clone ao PDF.

```
make pdf                # docs/relatorio/relatorio.md -> docs/relatorio/relatorio.pdf
```

O alvo `pdf` roda `pandoc` com `--pdf-engine=xelatex --toc --number-sections -V lang=pt-BR`, lendo `metadata.yaml` (título, autores, geometria A4, fontes) e `relatorio.md`; as figuras são os PNG de `docs/02-arquitetura/diagramas/`.

9 Perguntas prováveis da banca

Preparação para a apresentação. Cada resposta é curta, cita o número que a sustenta e aponta onde está o detalhe. Números do PixSim são do mundo B (confirmação) salvo indicação; custo é o ponderado por 1.000 Pix.

9.1 O problema e a decisão

Por que regras explícitas decidem, e não um modelo? Não há rótulo real de fraude no hackathon. Regra é explicável (o `reason` diz o porquê ao cliente e ao analista), reproduzível e auditável, e resolve o *cold start*. Os modelos existem, mas em *shadow*: calculam e gravam, nunca mudam a decisão (ADR 0017, 0022). No PixSim, regras + LightGBM custariam R$ 99,42 contra R$ 118,82 só com regras; isso só seria levado a produção com rótulo real.

Por que `rejected` não é erro HTTP? Reprovar é uma decisão de negócio sobre um Pix válido, não uma falha da requisição. A resposta é 201 com `status: rejected`; erro (4xx) fica para payload inválido, conflito de `id` (409, QIT002002) ou análise inexistente (404, QIT002001) (ADR 0003).

Como várias regras viram uma decisão? Dois 2FA não somam uma reprovação? Todas as regras rodam; vence a mais severa (`rejected` > `two_factor_required` > `approved`) e todos os disparos vão para `metadata.reasons`. Dois sinais de dúvida não viram reprovação: só uma tipologia explícita (conta tomada, mula) ou um sinal extremo reprova (ADR 0006, 0018).

Por que a maioria das regras pede 2FA em vez de reprovar? Pelo custo assimétrico: 2FA em legítimo custa R$ 2 de atrito; reprovar legítimo custa R$ 50 + 2% do valor; fraude com 2FA escapa em 10% dos casos (vítima coagida confirma). Com isso, reprovar só compensa quando o risco é claro (ADR 0030).

9.2 Regras e calibração

Como os parâmetros foram escolhidos? Grid sobre o backtest no PixSim, minimizando o custo ponderado: o mundo A escolhe, o mundo B confirma. A calibração tirou o custo de R$ 127,26 para R$ 118,82 no mundo B com as regras atuais. Parâmetro só muda com evidência do PixSim; o nosso simulador é só *fixture* do CI (ADR 0029).

Por que o limite noturno fica em R$ 1.000 se o PixSim preferia R$ 2.000? É a âncora regulatória (Res. BCB 142/2021; IN BCB 512/2024): R$ 1.000 por conta das 20h às 6h para pessoa física. Manter custa cerca de R$ 4,80 por 1.000 Pix no mundo B. Parte disso é artefato do gerador: o PixSim já limita a conta tomada à noite a R$ 1.000, então a regra não consegue disparar nela.

Por que três regras foram descartadas? A família "Montante" ficou sem regra? `TYPOLOGY_SPLITTING`, `COUNTERPARTY_FLAGGED` e `AMOUNT_SOURCE_DAILY_SUM` tinham ablação negativa nos dois mundos e nenhuma âncora regulatória. Desligadas juntas, o custo caiu de R$ 132,13 para R$ 109,61 (mundo A) e de R$ 153,51 para R$ 118,82 (B), com 1 ponto a menos de vítimas barradas. Sim: Montante ficou sem regra própria; o teto por Pix e o z robusto cobrem parte (ADR 0033).

Qual é a regra mais importante? `COUNTERPARTY_FIRST_TIME_HIGH` (primeiro Pix do pagador para aquele destino acima de R$ 1.000): +R$ 34,63 por 1.000 Pix; sem ela, a captura de vítimas cai de 51,1% para 27,7%. Depois vêm `PROFILE_NEW_DEVICE` (+R$ 5,60) e `VELOCITY_POISSON_GAMMA` (+R$ 3,77).

`COUNTERPARTY_NEW_ACCOUNT` custa R$ 25,38 a mais no PixSim. Por que fica? Porque o art. 39-B, §1º, II do Regulamento do Pix manda considerar o tempo de abertura da conta do recebedor. No PixSim as mulas têm idade comum, então a regra só pega legítimos que pagam contas novas; em Pix real, conta laranja recém-aberta é padrão documentado. É a decisão registrada: âncora regulatória vence o gerador.

E as regras que nunca disparam no PixSim? São cinco: rajada do pagador, fazenda de aparelhos, viagem impossível, funil de vítimas e reputação de grafo. O PixSim não gera esses sinais (não tem geolocalização, usa um aparelho por conta, espalha vítimas no tempo, e o worker não roda no backtest). Elas custam zero lá, têm testes pela API e são exercitadas na trava do CI. Não inventamos geolocalização no adaptador para não medir a nossa própria invenção.

9.3 Validação

Por que não validar com o próprio simulador? Seria circular: nós desenhamos a fraude e as regras que a pegam. No nosso simulador as regras barravam 98,5% das vítimas; no PixSim, 51%. Por isso o nosso simulador ficou só como trava de regressão do CI, e todo número com valor estatístico vem do PixSim (ADR 0029).

O PixSim é confiável? É um gerador de terceiros (Apache-2.0), calibrado com dados abertos do BCB, com tipologias que não desenhamos e o relógio do MED. Mas é sintético e várias formas são premissas declaradas (`ASSUMED`). Rodamos em escala reduzida (5.025 contas, 30 dias, uma seed por mundo), então diferenças abaixo de cerca de R$ 1 por 1.000 Pix são ruído.

Houve vazamento de dados (*data leakage*)? De rótulo, não. O adaptador só passa campos do contrato; as features usam só o histórico passado (paridade API × backtest com 0 divergências); os mundos A e B são independentes (só 3 de 5.025 contas com a mesma data de abertura; mulas com o mesmo id no nível do acaso). Os valores "colados" do PixSim (R$ 999,99…) são 22% das fraudes, mas o modelo não depende deles (AUC 0,992 → 0,990 sem eles). O que existe é viés: o mundo B foi usado na ablação, por isso o descarte foi confirmado também no A.

O AUC de 0,99 não é bom demais? É otimista, e dizemos isso: A e B vêm do mesmo gerador, só muda a seed. Só o valor do Pix já dá AUC 0,937, porque as medianas de fraude são premissas do PixSim. No sentido inverso (treino no PixSim, teste no nosso simulador) o LightGBM cai a 0,78 e as regras ganham.

Como vocês comparam com o artigo do PixSim? Mesma perspectiva (banco pagador), mesmos positivos (Pix de vítima), mesmo peso de importância e mesmo custo de falso bloqueio (R$ 50 + 2%). Rodando o relógio do MED do próprio PixSim sobre as nossas decisões: sem política se perde 79,3% do valor; com as regras, 20,7%; com regras + LightGBM, 9,0%. O `threshold_block` do PixSim perde 3,0%, mas com 485 falsos bloqueios contra 2 das nossas regras.

E se o modelo fosse pior, como um scorer real? Replicamos o experimento de ruído do artigo (logit + ruído gaussiano). Com AUC 0,82, o LightGBM com limiar reajustado custa R$ 127,08 contra R$ 118,82 das regras: sem ganho. O modelo só paga a fricção que causa com AUC acima de cerca de 0,93.

Como o motor protege contra engenharia social, se a própria vítima faz o Pix e pode confirmar o 2FA? Em boa parte, não protege: é o ponto mais fraco. Na engenharia social nenhum Pix de vítima é reprovado; no mundo A, 46,9% recebem 2FA e 53,1% passam. O custo assume que só 10% das vítimas coagidas confirmam o 2FA, premissa sem fonte. Com 30% o custo vai a R$ 139,88; acima de ~40% o motor custa mais do que aprovar tudo (R$ 151,29). O que ajuda hoje: primeiro Pix alto para destino novo, rajada fora do ritmo, limite noturno e, depois do fato, MED, bloqueio cautelar e o sinal de destino com fraude confirmada. O que ajudaria mais: 2FA como alerta específico, retenção do primeiro Pix alto para destino novo, teto para destinatário novo e o lado do recebedor (DICT, art. 39-B).

9.4 Modelos e retroalimentação

Como os modelos rodam no *hot path* sem scikit-learn na imagem? Os modelos são treinados offline e exportados para JSON (pesos e árvores); a inferência é Python puro. A diferença contra a biblioteca é de ~1e-16 (LightGBM) a ~1e-15 (regressão logística), verificada por `make model-parity` (ADR 0024).

Por que GraphSAGE e HMM ficam num worker separado? Grafo e sequência pedem numpy e uma visão de muitos Pix; isso não cabe nos milissegundos do POST. O worker calcula a cada 30 s e grava o score; o *hot path* só lê o último score com até 120 min (ADR 0025, 0026). O GraphSAGE tem AUC de nó 0,92 no mundo B, otimista porque o rótulo `is_mule` é estático.

Como fica o atraso do rótulo (MED)? Treinando só com as fraudes já reportadas (mediana de 2 dias, cauda de 17), o AUC cai pouco (0,989 → 0,968) e o custo sobe ~20%. Com um atraso fixo de 14 dias o modelo fica quase cego (AUC 0,66): os positivos recentes, os mais úteis, viram "legítimos" (Dal Pozzolo et al., 2018).

Como o banco aprenderia com os desfechos? Simulamos no backtest, não na API (ADR 0032). O sinal "destino com fraude já confirmada no MED" é o ganho mais barato: R$ 118,82 → R$ 112,23 e captura de 51% → 65%. O retreino semanal com *champion/challenger* promoveu dois desafiantes que pioraram depois (R$ 103,18 contra R$ 100,84 do modelo fixo): uma semana de sombra não basta; em produção, exigiríamos margem ou mais tempo.

O que é o viés de seleção? Pix reprovado não liquida e não gera desfecho. Com o modelo decidindo, 77% das fraudes nunca geram rótulo, e a precisão do 2FA vista pelos desfechos fica ~10 vezes abaixo da real. Monitorar só por rótulo subestima o motor.

9.5 Engenharia

Qual a latência e quanto aguenta? O motor avalia um Pix em P50 0,33 ms e P95 0,60 ms (backtest em memória, ~2,7 mil Pix/s). Na API, o custo dominante são as consultas de janela no Postgres (índices por pagador, aparelho e destino). A API é sem estado e escala horizontalmente; não fizemos teste de carga, então não prometemos um número de Pix/s em produção.

Como garantem que o backtest decide igual à API? O motor só conhece o `HistoryPort`: em produção, SQL; no backtest, índices em memória. `make parity` manda o mesmo roteiro pelos dois e compara status, regras e as 33 features: 0 divergências (ADR 0023).

Por que o "agora" é o `transaction_date` e não o relógio do servidor? Para a decisão ser reproduzível: o mesmo Pix com o mesmo histórico sempre recebe a mesma resposta, no teste, no backtest e na reanálise (ADR 0007).

E idempotência? O mesmo `id` com o mesmo payload devolve a análise gravada (200). O mesmo `id` com payload diferente é conflito (409, QIT002002). Todo Pix é gravado, inclusive os reprovados; as tabelas são *append-only* e dinheiro é inteiro em centavos.

Como testam? 113 testes *black box* pela API, sem importar `src/` nem ler o banco: cada regra tem um teste que não dispara e um que dispara, mais cenários, contrato e idempotência. O CI roda os testes, a paridade e a trava de calibração; o lint roda no `make check` antes de cada commit.

9.6 Próximos passos e processo

O que fariam com dados reais? (1) Rota de desfechos (`POST /pix/transaction/{id}/outcome`) para receber o MED; (2) consulta às notificações de infração do DICT no lugar da reputação derivada; (3) recalibrar no histórico real e só então tirar um modelo de *shadow*, com *champion/challenger* exigindo margem; (4) teste de carga.

Como usaram IA no desenvolvimento? Como ferramenta de engenharia, com regras explícitas no repositório: `AGENTS.md` (limites, TDD, camadas), *skills* por tarefa e um CHANGELOG em que toda

mudança de agente é marcada como tal. Decisões de produto (descartar regras, manter o limite noturno, o que é validação) foram do time e estão nas ADRs.

Referências

[1] QI Tech. *Desafio Antifraude Pix* e slides de abertura do Hackathon QI Tech 2026 (Risk Solutions). Transcrição em `docs/01-contexto/enunciado.md`; PDFs em `docs/referencia/enunciado/`.

[2] Pesquisa QITech (documento interno do time, literatura branca e cinzenta). `docs/pesquisa/Pesquisa QITech.pdf`.

[3] Bolton, R. J.; Hand, D. J. (2002). *Statistical Fraud Detection: A Review*. Statistical Science, vol. 17.

[4] Pizzolato et al. (2025). *A Taxonomy of Pix Fraud in Brazil*. arXiv:2511.20902. https://arxiv.org/abs/2511.20902

[5] Zeimarani, B.; Khatib, A.; Mousavinasr, S.; Figueiredo, C. M. S. (2026). *PixSim: a calibrated open-source simulator of instant-payment fraud, recovery and interdiction under analyst capacity constraints*. Identificador arXiv ainda não publicado (TODO no `CITATION.cff` do repositório; não verificado). Código: https://github.com/BashirZeimarani/pixsim_public (Apache-2.0; commit `7fdfc90013960f46df60c3ea01838e768e7a420a`).

[6] Dal Pozzolo, A.; Caelen, O.; Johnson, R. A.; Bontempi, G. (2015). *Calibrating Probability with Undersampling for Unbalanced Classification*. IEEE SSCI 2015.

[7] Dal Pozzolo, A. et al. (2018). *Credit card fraud detection: a realistic modeling and a novel learning strategy*. IEEE Transactions on Neural Networks and Learning Systems, 29(8).

[8] Dal Pozzolo, A. et al. (2014). *Learned lessons in credit card fraud detection from a practitioner perspective*. Expert Systems with Applications, 41(10).

[9] Lucas, Y. et al. (2019). *Multiple perspectives HMM-based feature engineering for credit card fraud detection*. ACM SAC 2019.

[10] Carcillo, F. et al. (2018). *Scarff: a scalable framework for streaming credit card fraud detection with Spark*. Information Fusion 41, 182-194. doi:10.1016/j.inffus.2017.09.005

[11] Banco Central do Brasil. Resolução BCB nº 142/2021, art. 2º, I (limite noturno de R$ 1.000, 20h às 6h, pessoa física), hoje consolidado na Instrução Normativa BCB nº 512/2024, art. 3º, §3º e §7º, II.

[12] Banco Central do Brasil. Regulamento do Pix, arts. 37 e 39-B, §1º, I a V (fatores de avaliação de suspeita, como tempo desde a abertura da conta e recorrência; o 39-B trata do PSP do recebedor).

[13] Documentação e decisões do projeto: `docs/02-arquitetura/decisoes/` (ADRs 0001 a 0033), `docs/05-motor/` (regras, calibração, validação no PixSim, análises) e `scripts/backtest/cost-model.json`.

[14] Iglewicz, B.; Hoaglin, D. C. (1993). *How to Detect and Handle Outliers*. ASQC Quality Press (escore z modificado com mediana e MAD; resumo no NIST/SEMATECH e-Handbook, seção 1.3.5.17).

[15] Platt, J. C. (1999). *Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods*. Em *Advances in Large Margin Classifiers*, 61-74.

[16] Ke, G. et al. (2017). *LightGBM: a highly efficient gradient boosting decision tree*. NeurIPS 30, 3146-3154.

[17] Liu, F. T.; Ting, K. M.; Zhou, Z.-H. (2008). *Isolation forest*. IEEE ICDM, 413-422. doi:10.1109/ICDM.2008.17

[18] Hamilton, W. L.; Ying, R.; Leskovec, J. (2017). *Inductive representation learning on large graphs*. NeurIPS 30, 1024-1034.

[19] Rabiner, L. R. (1989). *A tutorial on hidden Markov models and selected applications in speech recognition*. Proceedings of the IEEE 77(2), 257-286.

[20] Gelman, A.; Carlin, J. B.; Stern, H. S.; Dunson, D. B.; Vehtari, A.; Rubin, D. B. (2013). *Bayesian Data Analysis*, 3ª ed. CRC Press.